

THE HAVEN PROTOCOL

CORE PROTOCOL BLUEPRINT

CPB — DOCUMENT 01

STATELESS HEALTHCARE COORDINATION INFRASTRUCTURE

Enterprise Interoperability • Cryptographic State Coordination • Economic Governance • Multi-Chain Routing

Technical Architecture Specification

Version 2.0

August 2026

ARCHITECTURAL CLASSIFICATION

Principal Systems Architecture
Protocol Engineering
Enterprise Clinical Interoperability
EVM Settlement Infrastructure
Cross-Chain Coordination

DESIGNATION

THE HAVEN PROTOCOL

A stateless coordination rail designed to operate beneath fragmented healthcare software networks without assuming centralized ownership of participating clinical databases.

CONFIDENTIAL INSTITUTIONAL Technical Document. This architecture suite is published openly on GitHub and is licensed strictly under the terms of the GNU Affero General Public License (AGPL-3.0). Unauthorized derivative commercial execution or private re-licensing without an enterprise waiver from the author is strictly prohibited.

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

PROTOCOL DEFINITION

1.1 Architectural Mandate

The Haven Protocol is a **stateless, asynchronous coordination and governance protocol for fragmented healthcare software environments**.

The protocol is positioned beneath participating healthcare applications rather than replacing them.

Its primary function is to establish a deterministic coordination rail across independently operated healthcare systems, including:

- Electronic Medical Record systems
- Laboratory Information Systems
- Pharmacy systems
- Hospital Information Systems
- HMO platforms
- Claims clearing systems
- Payment and point-of-sale infrastructure
- Diagnostic networks
- Clinical referral systems
- Healthcare applications operating across heterogeneous technical environments

Haven does not establish a centralized clinical database.

Haven does not require participating institutions to surrender ownership of their underlying records.

Haven does not require participating systems to migrate their existing databases into a common repository.

Instead, Haven establishes a **shared protocol state machine** through which participating systems can independently determine whether a cross-boundary event is:

1. authenticated;
2. structurally valid;
3. cryptographically attributable;
4. economically settled;
5. synchronized with the protocol;
6. eligible for downstream translation.

The protocol therefore treats interoperability primarily as a **state-coordination problem**, rather than as a conventional data-transfer problem.

1.2 GOVERNING ARCHITECTURAL INVARIANT

The principal Haven invariant is:

NO VALID PROTOCOL STATE → NO VALID CROSS-NETWORK STATE TRANSITION.

A participating healthcare organization may continue operating its internal applications, databases, billing systems, clinical workflows, and proprietary infrastructure independently.

However, a state transition requiring Haven-mediated interoperability must satisfy the protocol's validation envelope before translation across the designated trust boundary is permitted.

The logical condition is:

$$\begin{aligned} E_{\text{cross}}=1 &\Leftrightarrow \\ S_{\text{protocol}}=\text{VALID} \wedge P_{\text{settlement}}=\text{VALID} \wedge A_{\text{origin}}=\text{AUTHORIZED} \wedge I_{\text{event}}=\text{VALID} & \\ 1 \text{ \textit{iff} } S_{\text{protocol}}=\text{VALID} \wedge P_{\text{settlement}}=\text{VALID} \wedge A_{\text{origin}}=\text{AUTHORIZED} & \\ \wedge I_{\text{event}}=\text{VALID} & \end{aligned}$$

Where:

E_{cross}

represents authorization for a cross-network state transition.

S_{protocol}

represents the protocol synchronization state.

$P_{\text{settlement}}$

represents the associated settlement state.

A_{origin}

represents authenticated authority of the originating system.

I_{event}

represents event integrity.

This creates the foundational separation:

Clinical data remains locally authoritative.

Protocol state remains independently verifiable.

Cross-system coordination becomes cryptographically attributable.

1.3 SYSTEM BOUNDARY

The Haven architecture is intentionally divided into independent trust domains.

DOMAIN A — PARTICIPATING HEALTHCARE SYSTEM

The institution retains authority over:

- patient records
- clinical documentation
- internal identifiers
- local database schemas
- clinical workflows
- institutional applications
- internal access-control systems

Haven does not become the system of record.

DOMAIN B — HAVEN EDGE ADAPTER

A lightweight edge component operates at the institution's interoperability boundary.

Its responsibilities include:

- event normalization;
- schema translation;
- event-envelope construction;
- cryptographic signing;
- asynchronous transmission;
- acknowledgement processing;
- synchronization verification;
- protocol-policy enforcement;
- local circuit-breaker execution.

The Edge Adapter does not become the institutional database.

It acts as the controlled translation boundary between the institution and the Haven protocol.

DOMAIN C — HAVEN COORDINATION LAYER

The coordination layer processes protocol events without requiring permanent custody of the originating clinical payload.

A protocol event is represented conceptually as:

$$E = (ID, A, P, C_{prev}, C_{next}, T, R) \quad \mathcal{E} = (ID, A, P, C_{prev}, C_{next}, T, R)$$

Where:

- IDID = unique event identifier
- AA = authenticated originating authority
- PP = protocol policy context
- C_{prev} = commitment to the preceding state
- C_{next} = commitment to the resulting state
- TT = temporal metadata
- RR = settlement/reference metadata

The underlying clinical payload may remain within the originating institution.

DOMAIN D — EVM ATTESTATION AND SETTLEMENT LAYER

Polygon functions as the initial EVM settlement environment.

The ledger is used for protocol-relevant commitments, economic settlement, synchronization authorization, and forensic verification.

Raw clinical records are not required to reside on-chain.

The governing rule is therefore:

$$\text{Clinical Payload} \neq \text{Ledger Payload} \quad \boxed{\text{Clinical Payload} \neq \text{Ledger Payload}}$$

The ledger should receive only the minimum information necessary to establish the required cryptographic and economic properties.

DOMAIN E — CROSS-CHAIN ROUTING LAYER

Haven maintains a provider-agnostic interoperability abstraction.

The protocol's logical message is separated from the underlying transport provider:

$$\text{HavenEvent} \rightarrow \text{RoutingPolicy} \rightarrow \text{TransportAdapter} \rightarrow \text{DestinationChain} \quad \text{HavenEvent} \rightarrow \text{RoutingPolicy} \rightarrow \text{TransportAdapter} \rightarrow \text{DestinationChain}$$

This prevents the Haven state machine from becoming permanently dependent upon a single interoperability provider.

LayerZero and Chainlink CCIP can therefore operate as transport mechanisms beneath the Haven routing abstraction rather than defining the Haven protocol itself.

DOMAIN F — TREASURY AND GOVERNANCE

The economic control plane contains:

- protocol settlement contracts;
- fee-routing logic;
- corporate treasury destinations;
- secure vault destinations;
- multisignature authorization;
- hardware-backed key custody;
- governance controls;
- emergency controls;
- audit interfaces;
- forensic event records.

Treasury authority is cryptographically separated from clinical data authority.

A treasury administrator must not possess implicit authority to modify clinical state.

A clinical integration administrator must not possess implicit authority to modify treasury settlement logic.

1.4 THE STATELESS DESIGN PRINCIPLE

Haven deliberately avoids requiring a centralized database containing every participating institution's complete clinical history.

Instead, the protocol operates on **state commitments and event transitions**.

Let a local system contain clinical state:

$D_i(t)D_{-i}(t)$

At time t .

Haven does not require ownership of $D_i(t)D_{-i}(t)$.

Instead, the institution produces a commitment:

$C_i(t)=H(D_i(t))C_{-i}(t)=H(D_{-i}(t))$

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

where HH represents a cryptographic hash function.

When a valid event produces a new state:

$$D_i(t+1)=F(D_i(t),E_t) \quad D_{-i}(t+1)=F(D_{-i}(t),E_t)$$

the institution produces:

$$C_i(t+1)=H(D_i(t+1)) \quad C_{-i}(t+1)=H(D_{-i}(t+1))$$

Haven therefore needs to establish the relationship:

$$C_i(t) \rightarrow E_t \rightarrow C_i(t+1) \quad C_{-i}(t) \rightarrow E_t \rightarrow C_{-i}(t+1)$$

rather than storing the entire underlying clinical state.

This distinction is fundamental to the protocol.

Haven coordinates **proof of state transition**.

It does not require centralized ownership of the **state itself**.

1.5 CORE OBJECTIVE

The objective of the Core Protocol is consequently defined as:

Distributed State Coordination+Independent Verification+Economic Enforcement\boxed{ Distributed\ State\ Coordination + Independent\ Verification + Economic\ Enforcement }

without requiring:

Centralized Clinical Data CustodyCentralized\ Clinical\ Data\ Custody

The protocol is therefore designed to operate as an infrastructure layer beneath fragmented healthcare networks.

Its purpose is not to make every healthcare system identical.

Its purpose is to make independently operated healthcare systems **capable of recognizing, validating, and continuing authorized state transitions across institutional boundaries**.

MACRO ARCHITECTURE LAYOUT

2.1 Architectural Topology

The Haven Protocol shall be implemented as a **distributed coordination topology** composed of independent local systems, edge enforcement nodes, protocol coordination services, ledger settlement infrastructure, and optional cross-chain transport networks.

The canonical logical topology is:

Local Systems→Haven Edge→Protocol Event Fabric→EVM Settlement→Synchronization Attestation→Destination Edge
 $\boxed{\text{Local} \setminus \text{Systems} \rightarrow \text{Haven} \setminus \text{Edge} \rightarrow \text{Protocol} \setminus \text{Event} \setminus \text{Fabric} \rightarrow \text{EVM} \setminus \text{Settlement} \rightarrow \text{Synchronization} \setminus \text{Attestation} \rightarrow \text{Destination} \setminus \text{Edge}}$

The architecture deliberately avoids making any individual healthcare organization's database the authoritative source for inter-institutional coordination.

Instead, authority is decomposed across multiple technical boundaries.

2.2 SYSTEM COMPONENT MODEL

The protocol consists of the following principal components.

COMPONENT 01 — SOURCE SYSTEM

The originating healthcare application generates a local business event.

Examples include:

PatientRegistered PatientRegistered LabResultFinalized LabResultFinalized
PrescriptionIssued PrescriptionIssued ReferralCreated ReferralCreated
ClaimSubmitted ClaimSubmitted PaymentCompleted PaymentCompleted

The source system remains authoritative over the underlying transaction that occurred within its own environment.

COMPONENT 02 — EVENT NORMALIZATION ENGINE

The Haven Edge Adapter converts heterogeneous local representations into a canonical Haven event envelope.

For example:

FHIR/HL7/SQL/API/Event→Canonical Haven Event
FHIR/HL7/SQL/API/Event \rightarrow
Canonical\ Haven\ Event

The normalization layer must preserve semantic meaning while removing implementation-specific dependencies.

A source system may therefore use:

FHIR
HL7
REST
SOAP
SQL
GraphQL
Proprietary APIs
Message Queues

while Haven receives a normalized protocol representation.

The protocol does not require participating organizations to adopt a single internal application architecture.

2.3 EVENT ENVELOPE

Every Haven event shall be represented conceptually as:

$E = \{EID, SID, AID, TYPE, SEQ, C_{prev}, C_{next}, TS, POLICY, SETTLE, SIG\}$
 $\mathcal{E} = \{EID, SID, AID, TYPE, SEQ, C_{prev}, C_{next}, TS, POLICY, SETTLE, SIG\}$

Where:

EID

Globally unique event identifier.

SID

Source-system identifier.

AID

Authenticated institutional authority identifier.

TYPE

Canonical event class.

SEQ

Monotonic sequence reference.

CprevC_{prev}

Cryptographic commitment to the immediately preceding recognized state.

CnextC_{next}

Cryptographic commitment to the resulting state.

TS

Event timestamp.

POLICY

Applicable Haven routing and authorization policy.

SETTLE

Economic settlement reference.

SIG

Cryptographic signature generated by the authorized source.

The underlying clinical payload is not required to accompany every protocol-level event.

2.4 EDGE TRANSLATION BOUNDARY

The Edge Adapter constitutes the first controlled Haven enforcement boundary.

Its internal execution sequence is:

Receive→Validate→Normalize→Hash→Sign→Settle→Transmit→Await Attestation
Receive
→ Validate → Normalize → Hash → Sign → Settle
→ Transmit → Await Attestation

The adapter must operate asynchronously.

A clinical application must not be required to maintain an open synchronous network transaction while the complete Haven coordination lifecycle executes.

The adapter therefore decouples:

Clinical WorkflowClinical\ Workflow

from:

Protocol CoordinationProtocol\ Coordination

while preserving causal ordering.

2.5 ECONOMIC ENFORCEMENT BOUNDARY

The payment rail is treated as a protocol infrastructure dependency rather than an optional client-side software feature.

The governing model is:

Event→Settlement→SynchronizationEvent \rightarrow Settlement \rightarrow Synchronization

not:

Event→Synchronization→Optional PaymentEvent \rightarrow Synchronization \rightarrow Optional\ Payment

For a billable Haven event:

Pvalid=1P_{valid}=1

must be established through the designated settlement architecture before the event becomes eligible for synchronization.

The protocol tax is therefore an **infrastructure-level settlement obligation**, not a JavaScript variable, SDK flag, hidden checkout parameter, or client-controlled configuration value.

2.6 EVM SETTLEMENT SUBSYSTEM

The Polygon EVM environment provides the initial settlement substrate.

The settlement subsystem contains:

POS→Payment Endpoint→Immutable Splitter→Treasury/VaultPOS \rightarrow Payment\ Endpoint \rightarrow Immutable\ Splitter \rightarrow Treasury/Vault

The client-facing application is not the authoritative settlement mechanism.

The immutable contract is.

Consequently:

ClientConfig≠SettlementTruthClientConfig \neq SettlementTruth

while:

OnChainSettlementState=SettlementTruthOnChainSettlementState = SettlementTruth

subject to verification of the relevant chain, contract address, transaction status, and required confirmations/finality policy.

2.7 SPLITTER CONTRACT BOUNDARY

The designated settlement contract receives the protocol fee and applies deterministic allocation logic.

Conceptually:

$$F_{\text{total}} = F_{\text{treasury}} + F_{\text{haven}} \quad F_{\text{total}} = F_{\text{treasury}} + F_{\text{haven}}$$

For a 95/5 allocation:

$$F_{\text{treasury}} = 0.95 F_{\text{total}} \quad F_{\text{treasury}} = 0.95 F_{\text{total}} \quad F_{\text{haven}} = 0.05 F_{\text{total}} \quad F_{\text{haven}} = 0.05 F_{\text{total}}$$

For a fixed Haven protocol allocation:

$$F_{\text{haven}} = F_{\text{fixed}} \quad F_{\text{haven}} = F_{\text{fixed}}$$

and:

$$F_{\text{treasury}} = F_{\text{total}} - F_{\text{fixed}} \quad F_{\text{treasury}} = F_{\text{total}} - F_{\text{fixed}}$$

The contract must enforce these calculations independently of client-side code.

No client application should be capable of altering the allocation through a request parameter.

2.8 TREASURY SECURITY DOMAIN

Treasury destinations shall be controlled through a hardware-backed multisignature architecture.

The intended control hierarchy is:

Protocol Contract → Safe Multisig → Hardware Signers → Corporate Treasury
Protocol \ Contract
Safe \ Multisig \ Hardware \ Signers \ Corporate \ Treasury

Private keys should not be embedded within:

- application servers;
- Edge Adapters;
- browser applications;
- checkout code;
- CI/CD pipelines;

- cloud environment variables;
- source repositories.

The Edge Adapter must never possess unilateral authority over treasury funds.

2.9 SYNCHRONIZATION AUTHORITY

The protocol synchronization state represents whether a participating network remains eligible for Haven-mediated coordination.

Conceptually:

$\text{Sync}_i = \begin{cases} 1 & \text{if required protocol conditions are satisfied} \\ 0 & \text{otherwise} \end{cases}$

The synchronization state must be derived from authoritative protocol evidence rather than client assertions.

The adapter periodically queries the authoritative protocol state through a configured JSON-RPC provider.

The intended operational interval is:

$T_{\text{poll}} = 60 \text{ seconds}$

A failed verification must not be interpreted as proof of fraud.

Instead, the adapter should distinguish:

$\text{UNVERIFIED} \neq \text{INVALID}$

This distinction is critical to preventing an unreliable RPC provider from accidentally creating a false payment violation.

2.10 CIRCUIT-BREAKER MODEL

The adapter shall operate under a fail-closed interoperability policy.

The logical state machine is:

$\text{ACTIVE} \rightarrow \text{VERIFICATION} \rightarrow \text{AUTHORIZED}$
 $\text{ACTIVE} \rightarrow \text{VERIFICATION} \rightarrow \text{AUTHORIZED}$

or:

ACTIVE→VERIFICATION→UNAUTHORIZED→CIRCUIT BREAK
ACTIVE \rightarrow VERIFICATION \rightarrow UNAUTHORIZED \rightarrow CIRCUIT\ BREAK

The circuit breaker must prevent new Haven-mediated cross-network translations when authoritative synchronization requirements are demonstrably unsatisfied.

However, it must not silently destroy, delete, or corrupt locally stored clinical transactions.

Therefore:

Protocol Freeze≠Clinical Data Destruction
Protocol\ Freeze \neq Clinical\ Data\ Destruction

A local institution may continue recording its own clinical activity while Haven-mediated cross-boundary synchronization is suspended.

Pending events should remain durably queued for subsequent reconciliation according to the protocol's retry and recovery policy.

2.11 TRUST BOUNDARY SEPARATION

The macro architecture establishes five independent trust boundaries:

Boundary	Primary Authority
Clinical Record	Originating Institution
Event Signature	Authorized Source Identity
Settlement	Immutable EVM Contract
Treasury Custody	Multisignature Wallet
Cross-Network Transport	Haven Routing Policy

No single component should possess all five authorities.

This prevents architectural concentration of power.

The protocol therefore follows:

$Trust_{Haven} = \sum Independent\ Verifiable\ Assertions$
Trust_{Haven} = \sum Independent\ Verifiable\ Assertions

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

rather than:

$$\text{TrustHaven} = \text{TrustSingle ServerTrust}_{\{\text{Haven}\}} = \text{Trust}_{\{\text{Single} \backslash \text{Server}\}}$$

2.12 MACRO FAILURE PRINCIPLE

Any subsystem failure must be isolated to its corresponding architectural domain.

A Polygon RPC failure must not corrupt a clinical database.

A clinical database outage must not modify the treasury contract.

A compromised Edge Adapter must not acquire treasury signing authority.

A transport-provider failure must not invalidate the underlying local clinical state.

A failed cross-chain message must not imply deletion of the originating event.

The architecture therefore requires:

$$\text{Failure}_{\text{component}} \Rightarrow \text{Failure}_{\text{system}} \text{Failure}_{\{\text{component}\}} \not\rightarrow \text{Failure}_{\{\text{system}\}}$$

unless the failed component represents a deliberately designated **hard dependency** for the specific operation being attempted.

This isolation property is fundamental to production deployment.

STATELESS ASYNCHRONOUS EVENT TOKENIZATION

3.1 Definition

Haven shall represent a clinically relevant coordination occurrence as a **protocol event**, not as a centrally stored clinical record.

An event is an immutable assertion that a participating system transitioned from one locally authoritative state to another.

$$S_{i,t} \rightarrow E_k S_{i,t+1} \quad S_{\{i,t\}} \xrightarrow{E_k} S_{\{i,t+1\}}$$

The protocol records sufficient cryptographic evidence to verify this transition without assuming ownership of the underlying state.

3.2 Event Identity

Every event shall receive a globally unique identifier:

$$EID = H(N \parallel SID \parallel SEQ \parallel TS \parallel R) \quad EID = H(N \parallel SID \parallel SEQ \parallel TS \parallel R)$$

Where:

- NN = Haven protocol namespace/version
- SIDSID = source-system identifier
- SEQSEQ = source sequence number
- TSTS = creation timestamp
- RR = cryptographically secure nonce

The identifier must be deterministic for forensic reconstruction while maintaining sufficient entropy to prevent collision.

3.3 State Commitment

For local state D_t , the source system generates:

$$C_t = H(D_t) \quad C_t = H(D_t)$$

Following event E_t :

$$D_{t+1} = F(D_t, E_t) \quad D_{\{t+1\}} = F(D_{\{t\}}, E_{\{t\}})$$

The resulting commitment is:

$$C_{t+1} = H(D_{t+1}) \quad C_{\{t+1\}} = H(D_{\{t+1\}})$$

The Haven event therefore binds:

$$C_t \rightarrow E_t \rightarrow C_{t+1} \rightarrow E_{t+1} \rightarrow C_{t+2}$$

without requiring Haven to possess D_t or D_{t+1} .

3.4 Tokenization Meaning

The term **event tokenization** does not imply that clinical records are converted into transferable financial assets.

Within Haven, tokenization means converting an event into a **portable, cryptographically addressable protocol representation**.

The tokenized event contains:

$$TE = \{EID, C_{prev}, C_{next}, Authority, Sequence, Timestamp, Policy, SettlementRef, Signature\}$$
$$T_E = \{EID, C_{prev}, C_{next}, Authority, Sequence, Timestamp, Policy, SettlementRef, Signature\}$$

The clinical payload remains outside the protocol ledger unless separately authorized by the applicable architecture.

3.5 Asynchronous Execution

Haven events shall be processed asynchronously.

The originating application emits:

$$Event_{local} \rightarrow EdgeQueue \rightarrow Event_{local} \rightarrow EdgeQueue$$

The Edge Adapter subsequently performs:

$$Queue \rightarrow Normalize \rightarrow Commit \rightarrow Sign \rightarrow Settle \rightarrow Transmit \rightarrow Queue \rightarrow Normalize \rightarrow Commit \rightarrow Sign \rightarrow Settle \rightarrow Transmit$$

The originating clinical workflow therefore does not require synchronous completion of the entire interoperability transaction.

Each event receives an explicit lifecycle:

$$CREATED \rightarrow QUEUED \rightarrow SIGNED \rightarrow SETTLEMENT_VERIFIED \rightarrow SUBMITTED \rightarrow ATTESTED \rightarrow DELIVERED$$
$$CREATED \rightarrow QUEUED \rightarrow SIGNED \rightarrow SETTLEMENT_VERIFIED \rightarrow SUBMITTED \rightarrow ATTESTED \rightarrow DELIVERED$$

Failure states are explicit:

FAILEDFAILED REJECTEDREJECTED QUARANTINEDQUARANTINED
RECONCILIATION_REQUIREDRECONCILIATION_REQUIRED

No event may silently transition from failure to success without a new verifiable state transition.

3.6 Event Sequence Integrity

Each source system maintains a monotonically increasing sequence:

$$SEQ_{n+1} = SEQ_n + 1 \quad SEQ_{\{n+1\}} = SEQ_{n+1}$$

The adapter rejects:

$$SEQ_{incoming} \leq SEQ_{last} \quad SEQ_{\{incoming\}} \leq SEQ_{\{last\}}$$

unless the event is explicitly identified as a replay, retry, or reconciliation operation.

This establishes ordering without requiring Haven to maintain the source institution's complete transactional history.

3.7 Replay Protection

Every event is bound to:

$$DomainID + SourceID + Sequence + Nonce$$

A previously accepted event identifier cannot be accepted as a new state transition.

The protocol must maintain sufficient acceptance evidence to determine:

$$Accepted(EID) = TRUE$$

or:

$$Accepted(EID) = FALSE$$

This evidence may be represented through on-chain commitments, authenticated event indexes, or equivalent verifiable infrastructure without storing the underlying clinical payload.

3.8 Economic Binding

For billable events:

$E_{\text{billable}} \Rightarrow \text{SettlementRequired} \xrightarrow{\text{E_}} \text{SettlementRequired}$

The event therefore cannot become an authorized Haven cross-network transition solely because a client application asserts that the fee was paid.

The authoritative relationship is:

$\text{EventID} \leftrightarrow \text{SettlementReference} \leftrightarrow \text{ProtocolState} \quad \text{EventID} \xrightarrow{\text{left}} \text{SettlementReference} \xrightarrow{\text{right}} \text{ProtocolState}$

A client-side variable such as:

`feePaid = true`

has **zero protocol authority**.

Only independently verifiable settlement evidence may satisfy the economic condition.

3.9 Statelessness Constraint

Haven must not require:

$\text{HavenDB} \supseteq D_1 \cup D_2 \cup \dots \cup D_n \quad \text{HavenDB} \supseteq D_1 \cup D_2 \cup \dots \cup D_n$

Instead:

$\text{HavenState} = \{\text{Commitments}, \text{References}, \text{Attestations}, \text{Policies}\} \quad \text{HavenState} = \{\text{Commitments}, \text{References}, \text{Attestations}, \text{Policies}\}$

while:

$\text{ClinicalState}_i \in \text{Institution}_i \quad \text{ClinicalState}_i \in \text{Institution}_i$

Thus the protocol coordinates the relationship between states rather than becoming the owner of those states.

3.10 Core Invariant

For every accepted event:

$\text{Verify}(E_t) \Rightarrow \text{Verify}(C_t, E_t, C_{t+1}, \text{Authority}, \text{Settlement}) \boxed{\text{Verify}(E_t) \rightarrow \text{Verify}(C_t, E_t, C_{t+1}, \text{Authority}, \text{Settlement})}$

The protocol's objective is therefore not to reproduce the clinical record.

It is to make the **existence, integrity, ordering, authority, and settlement condition of the state transition independently verifiable.**

STATELESS ASYNCHRONOUS EVENT TOKENIZATION

3.11 CANONICAL EVENT ENVELOPE

The Haven Event Envelope is the minimum protocol object required to transport a validated state transition between independent systems.

Conceptually:

$$HE = H(\text{Version} \parallel \text{NetworkID} \parallel \text{SourceID} \parallel \text{EventID} \parallel \text{Sequence} \parallel \text{PreviousCommitment} \parallel \text{CurrentCommitment} \parallel \text{EventType} \parallel \text{Timestamp} \parallel \text{SettlementReference})$$
$$\mathcal{H}_E = H(\text{Version} \parallel \text{NetworkID} \parallel \text{SourceID} \parallel \text{EventID} \parallel \text{Sequence} \parallel \text{PreviousCommitment} \parallel \text{CurrentCommitment} \parallel \text{EventType} \parallel \text{Timestamp} \parallel \text{SettlementReference})$$

The envelope must contain no unnecessary clinical payload.

Its function is to establish an independently verifiable relationship between:

$$\text{OriginatingAuthority} \rightarrow \text{StateTransition} \rightarrow \text{Settlement} \rightarrow \text{DestinationOriginatingAuthority}$$
$$\rightarrow \text{StateTransition} \rightarrow \text{Settlement} \rightarrow \text{Destination}$$

3.12 EVENT TYPES

Haven events shall be classified independently from the software implementation that generated them.

Illustrative event classes include:

PATIENT_IDENTITY_ASSERTION PATIENT_IDENTITY_ASSERTION
CARE_ENCOUNTER CARE_ENCOUNTER LAB_RESULT LAB_RESULT
PRESCRIPTION PRESCRIPTION REFERRAL REFERRAL CLAIM CLAIM
PAYMENT PAYMENT DISPENSATION DISPENSATION
AUTHORIZATION AUTHORIZATION DISCHARGE DISCHARGE

The event taxonomy must remain extensible.

Adding a new event type must not require modification of the fundamental settlement or cryptographic architecture.

3.13 CONTENT ADDRESSING

The protocol shall use cryptographic commitments rather than centralized record identifiers as the primary integrity mechanism.

For payload DD:

$$C=H(D)C=H(D)$$

A receiving system possessing an authorized representation D'D' computes:

$$C'=H(D')C'=H(D')$$

and verifies:

$$C'=CC'=C$$

Therefore:

$$C'=C \Rightarrow \text{Integrity}(D',D)C'=C \Rightarrow \text{Integrity}(D',D)$$

subject to the security assumptions of the selected cryptographic hash.

Haven consequently does not need to inspect the complete clinical record to establish that the received representation corresponds to the committed state.

3.14 MERKLE AGGREGATION

High-volume events may be aggregated into Merkle structures.

Given event commitments:

$$C_1, C_2, \dots, C_n C_1, C_2, \dots, C_n$$

a Merkle root is constructed:

$$R=\text{MerkleRoot}(C_1, \dots, C_n)R=\text{MerkleRoot}(C_1, \dots, C_n)$$

The protocol can commit RR rather than individually committing every event to the settlement layer.

An individual event can subsequently prove inclusion through:

$$\text{Proof}(C_i, R)\text{Proof}(C_i, R)$$

This produces:

$$\text{Verify}(C_i, \text{Proof}, R) = \text{TRUE} \quad \text{Verify}(C_i, \text{Proof}, R) = \text{TRUE}$$

without requiring the verifier to possess the complete event set.

This mechanism materially reduces ledger-write requirements while preserving independently verifiable inclusion.

3.15 EVENT FINALITY

Haven shall distinguish three separate concepts:

$$\text{Submission} \neq \text{Confirmation} \neq \text{Finality} \quad \text{Submission} \neq \text{Confirmation} \neq \text{Finality}$$

Submission means the event has been transmitted to the designated protocol or settlement endpoint.

Confirmation means the receiving infrastructure has acknowledged processing.

Finality means the protocol's defined settlement and ordering conditions have been satisfied to the required security threshold.

An event must not be represented as economically final merely because a transaction hash exists.

3.16 IDEMPOTENT PROCESSING

Every event-processing component shall support idempotency.

For event EID: EID:

$$\text{Process}(EID, n) = \text{Process}(EID, 1) \quad \text{Process}(EID, n) = \text{Process}(EID, 1)$$

where repeated delivery produces no additional state mutation after the first accepted execution.

This is mandatory because asynchronous networks inherently permit:

- retries;
- duplicate messages;
- delayed acknowledgements;
- provider failover;
- network reconnection;
- queue replay.

The protocol must therefore assume duplicate delivery as a normal operating condition.

3.17 EVENT RECONCILIATION

When source and destination state commitments diverge:

$$C_{source} \neq C_{destination} \quad C_{source} \neq C_{destination}$$

the destination must not silently overwrite its existing state.

The event enters:

RECONCILIATION_REQUIRED RECONCILIATION_REQUIRED

The reconciliation process compares:

EventID	EventID	Sequence	Sequence	PreviousCommitment	PreviousCommitment
CurrentCommitment	CurrentCommitment	SettlementReference	SettlementReference		
Signature	Signature				

The objective is to determine whether the divergence resulted from:

1. transmission failure;
2. duplicate processing;
3. stale destination state;
4. schema translation error;
5. unauthorized mutation;
6. conflicting concurrent events.

3.18 CONCURRENCY CONTROL

Independent healthcare systems may generate events simultaneously.

Haven therefore cannot assume a globally synchronous event stream.

For concurrent events:

$$E_a \parallel E_b \quad E_a \parallel E_b$$

the protocol requires deterministic ordering or explicit conflict resolution.

Ordering may be established through:

Sequence + Timestamp + SourceAuthority + EventID
Sequence + Timestamp + SourceAuthority + EventID

where the final ordering rule is deterministic and independently reproducible.

A timestamp alone shall never constitute sufficient ordering authority.

3.19 STATE TRANSITION ACCEPTANCE

An event is accepted only when all mandatory predicates evaluate successfully:

$$\text{Accept}(E) = \text{Auth}(E) \wedge \text{Integrity}(E) \wedge \text{Order}(E) \wedge \text{Settlement}(E) \wedge \text{Policy}(E)$$
$$\text{Accept}(E) = \text{Auth}(E) \wedge \text{Integrity}(E) \wedge \text{Order}(E) \wedge \text{Settlement}(E) \wedge \text{Policy}(E)$$

Therefore:

$$\text{Accept}(E) = 0 \vee \text{Accept}(E) = 1$$

if any mandatory predicate fails.

The adapter must not convert an invalid protocol state into an apparently valid downstream clinical state.

3.20 EVENT LIFECYCLE INVARIANT

For every protocol event:

$$\text{Created} \rightarrow \text{Validated} \rightarrow \text{Committed} \rightarrow \text{Settled} \rightarrow \text{Transmitted} \rightarrow \text{Attested} \rightarrow \text{Applied}$$
$$\text{Validated} \rightarrow \text{Committed} \rightarrow \text{Settled} \rightarrow \text{Transmitted} \rightarrow \text{Attested} \rightarrow \text{Applied}$$

The reverse operation is not permitted.

An applied event cannot be retrospectively represented as never having existed.

Corrections therefore require a new event:

$$E_{\text{correction}} \{ \text{correction} \}$$

rather than destructive mutation of:

$$E_{\text{original}} \{ \text{original} \}$$

This creates an append-oriented forensic history while keeping the underlying clinical database under institutional control.

STATELESS ASYNCHRONOUS EVENT TOKENIZATION

3.21 FORMAL STATE REPRESENTATION

For each participating system i , define its local state at time t as:

$$S_i(t)$$

The state is controlled and persisted by the participating institution.

Haven maintains only a protocol representation:

$$\Pi_i(t) = \{CID_i(t), SEQ_i(t), AUTH_i(t), POLICY_i(t)\} \setminus \Pi_i(t) = \{CID_i(t), SEQ_i(t), AUTH_i(t), POLICY_i(t)\}$$

where:

- $CID_i(t)$ = cryptographic commitment to local state;
- $SEQ_i(t)$ = accepted event sequence;
- $AUTH_i(t)$ = authorized institutional identity;
- $POLICY_i(t)$ = applicable protocol policy.

Therefore:

$$\Pi_i(t) \supset S_i(t) \setminus \Pi_i(t) \not\supset S_i(t)$$

The protocol representation does not contain the underlying clinical state.

3.22 STATE TRANSITION FUNCTION

A valid event E_k transforms the local state according to:

$$S_i(t+1) = F_i(S_i(t), E_k) \quad S_i(t+1) = F_i(S_i(t), E_k)$$

The corresponding protocol commitment becomes:

$$CID_i(t+1) = H(CID_i(t) \parallel E_k \parallel M_k) \quad CID_i(t+1) = H(CID_i(t) \parallel E_k \parallel M_k)$$

where M_k represents mandatory event metadata.

The protocol consequently records the mathematical relationship between successive commitments rather than the complete states themselves.

3.23 STATELESSNESS PROPOSITION

Proposition 1 — Protocol Verifiability Without Clinical State Custody

Assume:

1. HH is collision resistant;
2. the originating institution controls its signing key;
3. $CID_t = H(St) \quad CID_{t+1} = H(S_{t+1})$;
4. event E_t is cryptographically bound to CID_t and CID_{t+1} ;
5. the verifier possesses the authorized state representation required for verification.

Then Haven can verify the consistency of the transition:

$$St \rightarrow Et \quad St+1S_t \xrightarrow{\{E_t\}} S_{t+1}$$

without permanently storing StS_t or $St+1S_{t+1}$.

Proof

The verifier computes:

$$H(St) = CID_t \quad H(S_t) = CID_t$$

and:

$$H(St+1) = CID_{t+1} \quad H(S_{t+1}) = CID_{t+1}$$

The event establishes:

$$CID_t \rightarrow Et \rightarrow CID_{t+1} \quad CID_t \xrightarrow{E_t} CID_{t+1}$$

If both commitments match the protocol commitments and the event signature is valid, the transition is cryptographically consistent.

Therefore, persistent centralized custody of StS_t and $St+1S_{t+1}$ is unnecessary for integrity verification.

QED\boxed{QED}

3.24 NON-CUSTODIAL STATE PROPOSITION

Define total clinical data across n institutions:

$$D = \bigcup_{i=1}^n D_i$$

A centralized architecture requires:

$$D \subseteq \text{CentralRepository} \setminus \text{CentralRepository}$$

Haven instead requires:

$$H(D_i) \rightarrow \text{Protocol} \setminus H(D_i) \rightarrow \text{Protocol}$$

while:

$$D_i \rightarrow \text{Institution}_i \setminus D_i \rightarrow \text{Institution}_i$$

Therefore:

$$\text{ProtocolStorage} \cap \text{ClinicalPayload} \approx \emptyset \setminus \text{ProtocolStorage} \cap \text{ClinicalPayload} \approx \text{nothing}$$

except where explicitly authorized metadata or encrypted payload transport is required by a particular deployment.

The protocol consequently maintains **coordination evidence**, not centralized clinical ownership.

3.25 EVENT CHAIN INTEGRITY

Let:

$$C_0$$

represent the initial recognized commitment.

For events:

$$E_1, E_2, \dots, E_n$$

the protocol derives:

$$C_1 = H(C_0 \parallel E_1) \quad C_2 = H(C_1 \parallel E_2) \quad C_3 = H(C_2 \parallel E_3) \quad \dots \quad C_n = H(C_{n-1} \parallel E_n)$$

and generally:

$$C_n = H(C_{n-1} \parallel E_n)$$

A modification to any prior event E_j produces:

$$C_j' \neq C_j$$

with overwhelming probability under the collision-resistance assumption.

Consequently, downstream commitments become inconsistent:

$$C_{j+1}' \neq C_{j+1} \quad C_{j+1}' \neq C_{j+1}$$

and ultimately:

$$C_n' \neq C_n \quad C_n' \neq C_n$$

The event chain therefore provides tamper-evident historical continuity.

3.26 FORWARD VERIFICATION

A destination system need not retrieve the entire historical clinical record to verify the latest protocol state.

Given:

$$C_{n-1} \quad C_{n-1}$$

and:

$$E_n \quad E_n$$

the verifier computes:

$$C_n' = H(C_{n-1} \parallel E_n) \quad C_n' = H(C_{n-1} \parallel E_n)$$

and tests:

$$C_n' = C_n \quad C_n' = C_n$$

The verification cost is therefore dependent primarily on the size of the verification material rather than the total historical clinical dataset.

Conceptually:

$$\text{Complexity}_{\text{verification}} \approx O(1) \quad \text{Complexity}_{\text{verification}} \approx O(1)$$

for a single commitment transition, excluding external transport and cryptographic signature verification costs.

3.27 MERKLE STATE COMPRESSION

For event set:

$$E = \{E_1, E_2, \dots, E_n\} \quad \mathcal{E} = \{E_1, E_2, \dots, E_n\}$$

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

the protocol may calculate:

$$R = \text{MerkleRoot}(E) \quad R = \text{MerkleRoot}(\mathcal{E})$$

A verifier possessing event E_k and its inclusion proof P_k can establish:

$$\text{Verify}(E_k, P_k, R) = \text{TRUE} \quad \text{Verify}(E_k, P_k, R) = \text{TRUE}$$

The protocol therefore achieves:

$$\text{Storage}_{\text{commitment}} \ll \text{Storage}_{\text{clinical}} \quad \text{Storage}_{\text{commitment}} \ll \text{Storage}_{\text{clinical}}$$

while maintaining verifiable inclusion.

This mechanism is particularly applicable to high-volume laboratory, pharmacy, claims, and point-of-sale event streams.

3.28 INFORMATION MINIMIZATION

The protocol shall apply a strict minimization rule:

$$\text{Transmit}(\text{Data}) = \text{Minimum}(\text{Data}) \text{ required for the authorized state transition} \quad \text{Transmit}(\text{Data}) = \text{Minimum}(\text{Data}) \text{ required for the authorized state transition}$$

The presence of a data element inside a source system does not establish a requirement to transmit that element through Haven.

A destination receives only the information required by:

$$\text{Purpose} + \text{Authorization} + \text{Workflow} + \text{Policy} \quad \text{Purpose} + \text{Authorization} + \text{Workflow} + \text{Policy}$$

This creates a structural separation between:

$$\text{Data}_{\text{Availability}} \quad \text{Data}_{\text{Availability}}$$

and:

$$\text{Data}_{\text{Disclosure}} \quad \text{Data}_{\text{Disclosure}}$$

3.29 LIMITATION OF THE PROOF

The statelessness proof establishes **integrity and state-transition verifiability**.

It does not independently establish that the underlying clinical statement is medically correct.

If an institution signs an incorrect record:

$H(\text{FalseData})H(\text{FalseData})$

remains cryptographically valid.

Therefore:

$\text{CryptographicIntegrity} \neq \text{ClinicalTruth}$

Clinical correctness remains the responsibility of the originating authority and applicable governance controls.

This distinction is mandatory for the protocol's legal, clinical, and technical boundaries.

3.30 SECTION 3 TERMINATION CONDITION

Section 3 is satisfied when the protocol can establish:

$\boxed{\text{Authority} + \text{Integrity} + \text{Ordering} + \text{StateContinuity} + \text{SettlementReference}}$

without requiring centralized custody of the underlying clinical database.

The resulting architecture transforms interoperability from:

$\text{DataCopyingDataCopying}$

into:

$\text{VerifiableStateTransitionVerifiableStateTransition}$

This is the foundational mechanism upon which the subsequent economic enforcement and cross-network routing layers operate.

ZERO-TRUST PAYMENT ROUTING ENCLOSURE ENGINE

4.1 Purpose

The Zero-Trust Payment Routing Enclosure Engine (ZTPREE) is the economic enforcement subsystem of Haven.

Its purpose is to ensure that the protocol's mandatory infrastructure charge cannot be converted into an optional client application feature.

The governing principle is:

$\text{Payment Authorization} \neq \text{Client Assertion} \boxed{\text{Payment} \setminus \text{Authorization} \neq \text{Client} \setminus \text{Assertion}}$

A client application may request a protocol operation, but it cannot determine whether the associated economic obligation has been satisfied.

That determination must originate from independently verifiable settlement infrastructure.

4.2 TRUST MODEL

The payment architecture establishes the following hierarchy:

$\text{Client Software} < \text{Edge Adapter} < \text{Protocol Contract} < \text{Ledger State}$
 $\text{Client} \setminus \text{Software} < \text{Edge} \setminus \text{Adapter} < \text{Protocol} \setminus \text{Contract} < \text{Ledger} \setminus \text{State}$

The lower layer cannot override an authoritative condition established by the higher layer.

Therefore:

$\text{ClientConfig} \Rightarrow \text{PaymentValid}$
 $\text{ClientConfig} \not\Rightarrow \text{PaymentValid}$

and:

$\text{PaymentValid} \Leftarrow \text{VerifiedOnChainState}$
 $\text{PaymentValid} \not\Leftarrow \text{VerifiedOnChainState}$

The adapter must reject client-supplied variables purporting to establish payment completion.

Examples include:

feePaid
protocolFee
paymentVerified
syncEnabled
skipFee

bypassProtocol

Such variables may exist within a client application, but they possess no authority within the Haven security model.

4.3 PAYMENT PIPELINE

For a billable event, the canonical execution path is:

Checkout→ProtocolFee→SettlementEndpoint→ImmutableContract→Allocation→SettlementReference→Synchronization
Checkout → ProtocolFee → SettlementEndpoint
→ ImmutableContract → Allocation → SettlementReference
→ Synchronization

The protocol fee must therefore be bound to the transaction pipeline rather than implemented as an optional post-processing action.

The critical invariant is:

No Verified Settlement ⇒ No Billable Synchronization
 $\boxed{\text{No Verified Settlement} \rightarrow \text{No Billable Synchronization}}$

4.4 EVENT-TO-SETTLEMENT BINDING

Every billable protocol event must possess a settlement correlation mechanism.

Define:

$B(E) = H(EID \parallel Amount \parallel NetworkID \parallel ContractID)$
 $B(E) = H(EID \parallel Amount \parallel NetworkID \parallel ContractID)$

where $B(E)$ represents the event settlement binding.

The settlement transaction must expose sufficient verifiable information to establish correspondence between:

EID EID

and:

SettlementTx SettlementTx

The protocol should not rely solely on a wallet address or transaction existence because a valid transaction to the correct contract does not inherently prove that it corresponds to the specific protocol event under examination.

4.5 POLYGON SETTLEMENT DOMAIN

Polygon is the initial designated EVM settlement environment.

The settlement contract shall have a fixed protocol address represented conceptually as:

$C_{splitterC_}\{splitter\}$

The address becomes a protocol configuration constant for participating adapters.

The adapter must verify:

$chainId=Polygon$

and:

$contractAddress=C_{splitter}$

before accepting settlement evidence.

A transaction on another network does not satisfy Polygon settlement requirements.

Similarly:

$ContractAddress \neq C_{splitter}$

must result in rejection.

4.6 IMMUTABILITY REQUIREMENT

The designated production settlement contract shall not expose an administrative upgrade mechanism capable of changing:

- fee allocation;
- destination addresses;
- synchronization authorization semantics;
- settlement validation logic;
- event accounting rules.

Accordingly:

$Code_{t_0}=Code_{t_1}$

for all production states after deployment, subject only to the underlying blockchain's consensus assumptions.

An implementation that can be silently replaced by an administrator does not satisfy the strict interpretation of the Haven immutable-settlement requirement.

4.7 FEE ALLOCATION FUNCTION

For a total protocol charge:

F

the contract computes:

$$F = T + V$$

where:

- T = corporate treasury allocation;
- V = Haven vault allocation.

For a percentage allocation:

$$T = (1-r)F \quad V = rF$$

For the specified 95/5 model:

$$T = 0.95F \quad V = 0.05F$$

For a fixed allocation:

$$V = F_{\text{fixed}}$$

and:

$$T = F - F_{\text{fixed}}$$

The contract must perform this allocation deterministically.

No client-side calculation is authoritative.

4.8 TREASURY DESTINATION

The treasury destination shall be controlled by a Safe multisignature architecture with hardware-backed signing keys.

Conceptually:

Splitter→Safe→HardwareSigners→TreasurySplitter \rightarrow Safe \rightarrow
HardwareSigners \rightarrow Treasury

The Edge Adapter has no treasury signing authority.

The client application has no treasury signing authority.

A compromise of the client checkout environment must therefore not provide direct access to the corporate treasury.

4.9 SYNCHRONIZATION STATE

The protocol synchronization mapping is represented conceptually as:

mapping(address⇒bool)mapping(address\rightarrow bool)

with:

isNetworkSynchronized[A]isNetworkSynchronized[A]

representing the current protocol authorization state for network participant AA.

The state must be controlled by protocol-authorized contract logic.

The critical distinction is:

ClientClaims≠ContractStateClientClaims \neq ContractState

The adapter therefore reads the authoritative state from the designated contract through authenticated blockchain infrastructure.

4.10 CIRCUIT-BREAKER CONDITION

At the configured verification interval:

T_{check}=60sT_{check}=60s

the adapter evaluates the required synchronization conditions.

If:

$\text{isNetworkSynchronized}[A] = \text{TRUE} \wedge \text{isNetworkSynchronized}[A] = \text{TRUE}$

and the corresponding settlement conditions are valid, cross-network processing remains enabled.

If:

$\text{isNetworkSynchronized}[A] = \text{FALSE} \wedge \text{isNetworkSynchronized}[A] = \text{FALSE}$

the adapter transitions to:

$\text{CIRCUIT_OPEN} \rightarrow \text{CIRCUIT_OPEN}$

and prevents new Haven-mediated cross-network translations.

The local system remains operational.

The failure is isolated to the Haven interoperability boundary.

4.11 FAIL-CLOSED SECURITY PROPERTY

The adapter must not interpret ambiguous client-side responses as successful authorization.

The minimum decision model is:

$\text{Authorized} = \text{SyncState} \wedge \text{SettlementValid} \wedge \text{IdentityValid} \wedge \text{PolicyValid}$
 $\text{Authorized} = \text{SyncState} \wedge \text{SettlementValid} \wedge \text{IdentityValid} \wedge \text{PolicyValid}$

Therefore:

$\text{Authorized} = \text{FALSE} \wedge \text{Authorized} = \text{FALSE}$

whenever a mandatory condition is demonstrably false.

However, an RPC timeout must not automatically be treated as proof that:

$\text{SyncState} = \text{FALSE} \wedge \text{SyncState} = \text{FALSE}$

The adapter should enter:

VERIFICATION_DEGRADED\VERIFICATION_DEGRADED

until authoritative state can be established according to the deployment's configured availability policy.

This prevents a malicious or unreliable RPC endpoint from becoming an accidental denial-of-service mechanism.

4.12 ANTI-BYPASS PRINCIPLE

The economic enclosure is considered effective only if bypassing the client checkout path cannot produce an accepted Haven state transition.

The intended security relationship is:

$$\text{Bypass}(\text{ClientPayment}) \Rightarrow \text{NoSettlementProof} \Rightarrow \text{NoAuthorizedEvent} \Rightarrow \text{NoCrossNetworkTranslation}$$
$$\neg \text{Bypass}(\text{ClientPayment}) \Rightarrow \text{NoSettlementProof} \Rightarrow \text{NoAuthorizedEvent} \Rightarrow \text{NoCrossNetworkTranslation}$$

The protocol therefore does not attempt to prevent a developer from modifying their own software.

It makes such modification **insufficient to manufacture valid protocol state**.

This distinction is the core of the zero-trust payment architecture.

ZERO-TRUST PAYMENT ROUTING ENCLOSURE ENGINE

4.13 CONTRACT EXECUTION MODEL

The Haven settlement contract shall treat payment receipt and protocol authorization as two related but distinct state transitions.

Payment→Accounting→Authorization
Payment \rightarrow Accounting \rightarrow Authorization

A payment transaction alone must not automatically establish network synchronization unless the transaction contains sufficient authenticated information to identify the participating network and applicable protocol event.

The contract therefore operates on:

SettlementState=f(NetworkID,ParticipantID,EventReference,Amount,Destination,Nonce)
SettlementState = f(NetworkID, ParticipantID, EventReference, Amount, Destination, Nonce)

The contract's internal state must be deterministic and independently queryable.

4.14 receive() AND fallback() LIMITATION

The EVM provides two relevant Solidity entry mechanisms:

receive()
receive()

for plain native-token transfers with empty calldata, and:

fallback()
fallback()

for calls that do not match another function, including calls carrying calldata.

Neither mechanism should be treated as a generic parser capable of discovering information that was never transmitted by the transaction.

Therefore, the statement:

fallback()→Parse(PaymentContext)
fallback()\rightarrow Parse(PaymentContext)

is valid only when the required context is actually encoded in calldata.

A production Haven implementation shall therefore use an explicit settlement function where structured event metadata is required.

Conceptually:

`settle(ParticipantID,EventReference,Amount,Nonce)`
`settle( ParticipantID, EventReference, Amount, Nonce )`

rather than relying on an ambiguous fallback invocation.

The fallback path may remain disabled, rejected, or restricted to controlled compatibility behavior.

4.15 DETERMINISTIC SETTLEMENT RECORD

Each accepted settlement shall produce a protocol settlement record:

$SR = \{TxHash, BlockNumber, ParticipantID, EventReference, Amount, TreasuryAmount, VaultAmount, Nonce, ContractAddress\}$
 $SR = \{TxHash, BlockNumber, ParticipantID, EventReference, Amount, TreasuryAmount, VaultAmount, Nonce, ContractAddress\}$

The record creates the correlation:

$Event \leftrightarrow Settlement$

The adapter can subsequently verify the relationship without trusting the client application.

4.16 NONCE CONTROL

Every settlement request shall contain a unique protocol nonce.

For participant AA:

$Nonce_{A,n+1} > Nonce_{A,n}$

A previously consumed nonce must not be reusable.

The contract maintains sufficient state to enforce:

$UsedNonce(A,N) = TRUE$

after successful settlement.

A repeated submission:

(A,N)

must therefore fail or become an idempotent no-op according to the defined contract semantics.

4.17 FEE CONSERVATION INVARIANT

The contract shall preserve the total received protocol amount.

For every valid settlement:

$$F_{received} = F_{treasury} + F_{vault} + F_{remainder} \quad F_{received} = F_{treasury} + F_{vault} + F_{remainder}$$

where:

$$F_{remainder} = 0 \quad F_{remainder} = 0$$

unless an explicitly documented rounding mechanism applies.

Therefore:

$$F_{received} = F_{treasury} + F_{vault} \quad \boxed{F_{received} = F_{treasury} + F_{vault}}$$

The allocation function must not create or destroy value through arithmetic errors.

For integer-denominated EVM assets:

$$F_{treasury} = \lfloor (1-r)F \rfloor \quad F_{treasury} = \lfloor (1-r)F \rfloor$$

and:

$$F_{vault} = F - F_{treasury} \quad F_{vault} = F - F_{treasury}$$

This ensures conservation despite integer rounding.

4.18 SYNCHRONIZATION STATE TRANSITION

The synchronization flag shall not be treated as a permanent boolean assertion without a defined transition authority.

Conceptually:

$$Sync_A(t+1) = G(SettlementState_A, PolicyState_A, SecurityState_A) \quad Sync_A(t+1) = G(\\ SettlementState_A, PolicyState_A, SecurityState_A)$$

where AA represents the participating network.

A successful protocol settlement may establish eligibility.

A confirmed violation, expiration, administrative governance action, or defined security event may revoke eligibility.

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

The state machine is therefore:

UNSYNCHRONIZED $\leftrightarrow$ SYNCHRONIZEDUNSYNCHRONIZED \leftarrow
SYNCHRONIZED

subject to authenticated transition rules.

4.19 AUTHORITY SEPARATION

The system shall distinguish:

SettlementAuthoritySettlementAuthority

from:

SynchronizationAuthoritySynchronizationAuthority

and:

TreasuryAuthorityTreasuryAuthority

A treasury wallet should not automatically possess unrestricted authority to mutate synchronization state.

Similarly, a synchronization administrator should not possess unrestricted authority to withdraw treasury funds.

This implements least privilege at protocol level.

4.20 PAYMENT BYPASS ANALYSIS

Attack A — Client removes the fee calculation

If the client removes:

```
protocolFee = 0.05
```

the protocol does not accept the resulting client assertion.

The absence of valid settlement evidence produces:

SettlementValid=FALSESettlementValid=FALSE

Therefore:

Authorized=FALSEAuthorized=FALSE

Attack B — Client changes the destination wallet

If the client routes the fee to another address:

$Destination \neq C_{splitter}$

the transaction is not recognized as valid Haven settlement.

Therefore:

SettlementValid=FALSESettlementValid=FALSE

Attack C — Client fabricates `isNetworkSynchronized=true`

The adapter ignores client-side synchronization variables.

It queries the authoritative protocol state.

Therefore:

$ClientBoolean \notin TrustBoundary$

Attack D — Client replays an old valid transaction

The event's:

EventReferenceEventReference

and:

NonceNonce

are checked against previously accepted protocol state.

Therefore:

$Replay(E_{old}) \rightarrow REJECT$

Attack E — Client sends a valid payment unrelated to the event

A valid transaction does not automatically satisfy a different event.

The adapter must establish:

$EventReference = SettlementReference$

Therefore:

$ValidPayment \Rightarrow ValidEventSettlement$
without correlation.

4.21 RPC TRUST MODEL

The adapter shall not treat a single unauthenticated RPC response as absolute truth.

A production implementation should support:

$RPC_1, RPC_2, \dots, RPC_n$

or an equivalent independently monitored provider strategy.

The verification objective is:

$Verify(ChainID, ContractCode, ContractState, BlockReference, SettlementEvent)$

The adapter must also validate that the queried contract corresponds to the expected production deployment.

A malicious RPC response must not be capable of authorizing an otherwise invalid settlement.

4.22 60-SECOND VERIFICATION WINDOW

The nominal synchronization polling interval is:

$T = 60 \text{ seconds}$

This creates an operational maximum detection window under normal conditions of approximately:

$0 < T_{\text{detection}} \leq 60s$

excluding RPC latency, local scheduling delay, blockchain finality, network interruption, and recovery processing.

The 60-second value is therefore an **operational polling interval**, not a cryptographic guarantee of instantaneous revocation.

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

This distinction shall remain explicit throughout production documentation.

4.23 ENFORCEMENT LIMIT

The Edge Adapter is itself an enforcement point.

Therefore:

$\text{Compromise(Adapter)} \Rightarrow \text{Potential Local Bypass}$
 $\text{Compromise(Adapter)} \rightarrow \text{Potential Local Bypass}$

unless additional controls exist.

The architecture must consequently assume that a sufficiently privileged client can attempt to remove or replace the adapter.

The protocol's strongest security property is therefore not that client software is physically impossible to modify.

It is:

$\text{ModifiedClient} \Rightarrow \text{ValidProtocolState}$
 $\text{ModifiedClient} \not\rightarrow \text{ValidProtocolState}$

Additional deployment controls, signed adapter releases, remote attestation where appropriate, watchdog processes, immutable infrastructure, and contractual/network controls may be required where the threat model includes hostile operators with administrative control of the client environment.

ZERO-TRUST PAYMENT ROUTING ENCLOSURE ENGINE

4.24 SETTLEMENT CONTRACT SECURITY INVARIANTS

The Haven settlement contract shall satisfy the following invariants.

INVARIANT 01 — DESTINATION INTEGRITY

For every accepted settlement:

$\text{Destination}_{\text{treasury}} = \text{DT}_{\text{Destination_}\{\text{treasury}\}} = \text{D_T}$

and:

$\text{Destination}_{\text{vault}} = \text{DV}_{\text{Destination_}\{\text{vault}\}} = \text{D_V}$

where DT_T and DV_V are immutable or governance-constrained production destinations.

A client transaction parameter must not be capable of substituting either destination.

INVARIANT 02 — VALUE CONSERVATION

For every successful settlement:

$F_{\text{in}} = F_{\text{T}} + F_{\text{V}}_{\text{in}} = F_{\text{T}} + F_{\text{V}}$

subject only to explicitly defined integer rounding.

No undisclosed contract balance accumulation is permitted.

INVARIANT 03 — NONCE UNIQUENESS

For participant AA:

$\text{Used}(A, N) = \text{TRUE} \Rightarrow \text{Settlement}(A, N) \rightarrow \text{SUCCESS}$
 $\text{Used}(A, N) = \text{TRUE} \wedge \text{Settlement}(A, N) \not\rightarrow \text{SUCCESS}$

A valid settlement cannot be executed twice under the same protocol nonce.

INVARIANT 04 — EVENT CORRELATION

For every billable event:

$$EID \leftrightarrow \text{SettlementReference} EID \rightarrow \text{SettlementReference}$$

must be cryptographically or deterministically established.

A transaction cannot be reused to authorize unrelated protocol events.

INVARIANT 05 — AUTHORITY INTEGRITY

Only an authorized protocol authority may change synchronization state.

Therefore:

$$\text{UnauthorizedCaller} \rightarrow \text{StateMutation} = \text{FALSE} \text{UnauthorizedCaller} \rightarrow \text{StateMutation} = \text{FALSE}$$

INVARIANT 06 — CLIENT INDEPENDENCE

No client-supplied Boolean, fee value, wallet address, or synchronization flag shall override contract-derived settlement state.

$$\text{ClientInput} \supset \text{ProtocolAuthorityClientInput} \not\supset \text{ProtocolAuthority}$$

4.25 PROTOCOL FEE ACCOUNTING

The Haven economic model defines a protocol charge of:

$$F_{\text{protocol}} = \$0.05 F_{\{\text{protocol}\}} = \$0.05$$

or its designated local-currency equivalent, such as:

$$F_{\text{protocol}} = \text{₦}100 F_{\{\text{protocol}\}} = \text{₦}100$$

where the deployed pricing policy establishes the applicable conversion and collection mechanism.

The protocol charge is classified architecturally as an infrastructure pass-through operating expense.

The economic pipeline is:

CustomerCheckout→ProtocolCharge→SettlementContract→Treasury/VaultCustomerCheckout
↗ ProtocolCharge ↗ SettlementContract ↗ Treasury/Vault

The fee must therefore be represented as a discrete infrastructure line item rather than being silently deducted from an unrelated clinical charge.

4.26 PRICE-DENOMINATION SEPARATION

The protocol must distinguish:

ProtocolUnitProtocolUnit

from:

SettlementAssetSettlementAsset

and:

LocalCurrencyEquivalentLocalCurrencyEquivalent

For example:

$\$0.05 \neq \text{N}100 \neq \text{TokenAmount}$ $\$0.05 \neq \text{N}100 \neq \text{TokenAmount}$

unless the deployment's pricing policy explicitly establishes that equivalence.

A production implementation shall not hard-code a permanent FX assumption into immutable settlement logic unless the business model deliberately accepts the resulting economic risk.

Where local-currency collection is required while settlement occurs on Polygon, an off-chain payment processor or authorized settlement mechanism may convert the customer-facing charge into the designated blockchain settlement asset.

The resulting transaction must remain independently auditable.

4.27 CUSTOMER CHECKOUT BOUNDARY

The checkout application may display:

SubtotalSubtotal ClinicalServiceClinicalService HavenInfrastructureFeeHavenInfrastructureFee
TotalTotal

However, the checkout interface does not establish settlement validity.

The authoritative sequence is:

CheckoutPresentation→PaymentExecution→OnChainVerificationCheckoutPresentation
↗ PaymentExecution ↗ OnChainVerification

not:

CheckoutPresentation→PaymentVerifiedCheckoutPresentation ↗ PaymentVerified

A manipulated interface therefore cannot manufacture a valid protocol settlement.

4.28 CONTRACT ADDRESS PINNING

Each production Edge Adapter shall maintain an authenticated configuration containing the expected:

ChainIDChainID ContractAddressContractAddress ContractCodeHashContractCodeHash
DeploymentVersionDeploymentVersion

The adapter verifies that the deployed contract corresponds to the expected production artifact.

This creates:

ExpectedCodeHash=ObservedCodeHashExpectedCodeHash=ObservedCodeHash

as an additional integrity condition.

A contract deployed at the correct address but containing unexpected bytecode must not automatically be trusted.

4.29 CODE IMMUTABILITY VERIFICATION

For an immutable deployment:

CodeHasht=CodeHashproductionCodeHash_t=CodeHash_{production}

must remain true throughout the contract's operational lifetime.

Where a proxy architecture is used, the implementation address and upgrade authority become additional trust dependencies.

Because the Haven requirement specifies an **immutable non-upgradable splitter**, the preferred production configuration is a non-proxy deployment whose executable settlement logic cannot be replaced through an administrator-controlled upgrade mechanism.

4.30 TREASURY WITHDRAWAL SEPARATION

The settlement contract should minimize retained balances.

The preferred settlement pattern is:

Receive→Calculate→Transfer

rather than:

Receive→Accumulate→Manual Withdrawal

where practical.

This reduces:

$ContractBalance_{atRisk}$

and limits the financial consequence of a future contract vulnerability.

The treasury itself remains protected through multisignature custody.

4.31 FORMAL PAYMENT AUTHORIZATION

Define:

$P(E) = \begin{cases} 1 & \text{if settlement satisfies all protocol predicates} \\ 0 & \text{otherwise} \end{cases}$

Then:

$Sync(E) = P(E) \wedge A(E) \wedge I(E) \wedge O(E) \wedge Policy(E)$

Therefore:

$P(E)=0 \Rightarrow Sync(E)=0$

This is the formal economic gate.

Payment is not merely associated with interoperability.

It is one of the predicates required to authorize the protocol transition.

4.32 ENFORCEMENT CHAIN

The complete logical chain is:

Customer→Checkout→Settlement→Contract→OnChainEvidence→AdapterVerification→ProtocolAuthorization→CrossNetworkTranslation
Customer \rightarrow Checkout \rightarrow Settlement \rightarrow Contract \rightarrow OnChainEvidence \rightarrow AdapterVerification \rightarrow ProtocolAuthorization \rightarrow CrossNetworkTranslation

The client application therefore occupies the presentation and initiation layer.

It does not occupy the authoritative verification layer.

This is the central architectural distinction of ZTPREE.

4.33 SECURITY OBJECTIVE

The payment enclosure is considered operationally successful when the following implication holds:

UnauthorizedFeeBypass⇒UnauthorizedProtocolState⇒CrossNetworkDenial\boxed{UnauthorizedFeeBypass \rightarrow UnauthorizedProtocolState \rightarrow CrossNetworkDenial }

while:

AuthorizedSettlement⇒EligibleProtocolStateAuthorizedSettlement \rightarrow EligibleProtocolState

subject to all additional identity, integrity, ordering, policy, and security predicates.

The system is therefore designed so that **economic bypass does not produce interoperability privilege**.

ZERO-TRUST PAYMENT ROUTING ENCLOSURE ENGINE

4.34 ADVERSARIAL CONTROL MODEL

The payment enclosure shall be designed against an adversarial client assumption.

The protocol assumes that a participating developer may possess authority to modify:

- frontend code
- checkout logic
- SDK configuration
- API requests
- environment variables
- application middleware
- local database records

None of these authorities shall constitute protocol authority.

The security model therefore begins with:

TrustClient=MINIMUMTrust__{Client}=MINIMUM

and establishes authorization from infrastructure that exists outside the client's mutable application logic.

4.35 BYPASS RESISTANCE MATRIX

Attack	Expected Result
Remove fee from UI	Settlement absent; event rejected
Set fee to zero	Settlement absent/invalid
Change destination wallet	Contract mismatch; rejected
Forge payment Boolean	Ignored
Replay settlement	Nonce/EID rejection
Reuse settlement for another event	Correlation failure
Modify adapter configuration	Contract/address integrity failure
Query false RPC endpoint	Multi-source/integrity verification failure
Alter local database	Commitment divergence
Modify client SDK	No protocol authority gained

The architecture does not require the client application to be honest.

It requires the client application to be **unable to manufacture the evidence required for protocol authorization.**

INTERFACE DEFINITION & STORAGE LAYOUT INVARIANTS

4.36.1 CANONICAL SOLIDITY EVENT DEFINITIONS

All protocol settlement and circuit-breaker state transitions shall emit deterministic events suitable for independent off-chain indexing, monitoring, reconciliation, and forensic verification.

The canonical settlement event shall be:

```
event SettlementExecuted(  
    bytes32 indexed claimHash,  
    address indexed recipient,  
    uint256 amountSettled,  
    uint64 timestamp  
);
```

The canonical circuit-breaker event shall be:

```
event CircuitBreakerTripped(  
    uint16 indexed errorCode,  
    address indexed triggeredBy,  
    bytes32 stateHash  
);
```

Event emission shall occur only after the corresponding state transition has satisfied the contract's execution invariants.

Off-chain indexers shall treat emitted events as verifiable protocol telemetry and shall not treat event data alone as authority to modify protocol state.

4.36.2 EVENT INTEGRITY INVARIANT

For every settlement event EsE_s :

$SettlementExecuted \Rightarrow SettlementStateTransition = VALID$
 $SettlementExecuted \setminus \Rightarrow SettlementStateTransition = VALID$

For every circuit-breaker event EcE_c :

$CircuitBreakerTripped \Rightarrow CircuitState = TRIPPED$
 $CircuitBreakerTripped \setminus \Rightarrow CircuitState = TRIPPED$

An event shall not be emitted to represent a state transition that did not successfully occur.

4.36.3 IMMUTABLE SETTLEMENT CONTRACT STORAGE

The Polygon Revenue Splitter is designated as an immutable settlement component.

Accordingly, the production Splitter shall not depend upon an upgradeable proxy architecture unless the protocol specification is formally amended through governance.

The canonical architectural assumption is:

Splitter=NonUpgradeableSplitter = NonUpgradeable

Therefore:

ImplementationAddress=FixedImplementationAddress = Fixed

and:

ProxyUpgradeAuthority=NONEProxyUpgradeAuthority = NONE

for the immutable production settlement deployment.

4.36.4 STORAGE LAYOUT INVARIANT

Because the production Splitter is non-upgradable, a conventional upgradeability storage gap:

```
uint256[50] private __gap;
```

is not required as an upgrade mechanism.

The contract shall instead maintain an explicitly defined storage layout whose variables are fixed for the lifetime of the deployed implementation.

Any future change requiring a different storage layout shall require deployment of a new contract rather than mutation of the existing implementation.

Therefore:

StorageLayoutv1=ImmutableStorageLayout_{v1} = Immutable

and:

StorageLayoutv1→StorageLayoutv2StorageLayout_{v1}\rightarrow StorageLayout_{v2}

shall require:

NewContractDeploymentNewContractDeployment

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

rather than an in-place implementation upgrade.

4.36.5 STORAGE COLLISION INVARIANT

For every declared storage variable V_i :

$\text{Slot}(V_i)\text{Slot}(V_i)$

shall be uniquely determined at compilation and deployment.

The production contract shall not introduce dynamic implementation replacement that could reinterpret existing storage slots under a different contract layout.

Therefore:

$\text{ImplementationChange} \Rightarrow \text{StorageReinterpretation} \wedge \text{ImplementationChange} \not\Rightarrow \text{StorageReinterpretation}$

within the immutable Splitter deployment.

4.36.6 VERSIONED CONTRACT MIGRATION

Where a future protocol revision requires additional storage or altered settlement logic:

$\text{SplitterV1Splitter}_{\{V1\}}$

shall remain independently verifiable.

A new implementation shall be deployed as:

$\text{SplitterV2Splitter}_{\{V2\}}$

with a formally defined migration and route-activation procedure.

The existence of:

$\text{SplitterV2Splitter}_{\{V2\}}$

shall not silently modify the historical execution state of:

$\text{SplitterV1Splitter}_{\{V1\}}$

4.36.7 INTERFACE STABILITY

External protocol interfaces shall maintain deterministic function signatures and event definitions for the lifetime of a production contract deployment.

Any incompatible interface modification shall require:

VersionIncrementVersionIncrement

and:

NewContractDeploymentNewContractDeployment

The Edge Adapter shall reject an unexpected interface version.

4.36.8 INDEXING AND FORENSIC RECONSTRUCTION

The combination of:

EventLogsEventLogs TransactionHashTransactionHash BlockNumberBlockNumber
ContractAddressContractAddress

and:

EventParametersEventParameters

shall provide sufficient evidence for independent reconstruction of settlement and circuit-breaker execution history.

The indexing layer shall therefore function as an observability and forensic mechanism rather than as the authoritative source of protocol state.

4.36.9 CORE INTERFACE INVARIANT

The production interface architecture shall preserve:

ImmutableContract+DeterministicEvents+FixedStorageLayoutImmutableContract +
DeterministicEvents + FixedStorageLayout

without introducing an upgradeability mechanism that contradicts the immutable settlement requirement.

NoProxyUpgrade⇒NoImplementationSubstitution\boxed{ NoProxyUpgrade \rightarrow
NoImplementationSubstitution } NewProtocolVersion⇒NewDeployment\boxed{
NewProtocolVersion \rightarrow NewDeployment }

4.37 CIRCUIT-BREAKER STATE MACHINE

The adapter shall implement explicit operational states:

SYNCD
SYNCD DEGRADED
DEGRADED UNSYNCD
UNSYNCD
CIRCUIT_OPEN
CIRCUIT_OPEN

SYNCD

All mandatory protocol conditions are verified.

Cross-network translation is permitted.

DEGRADED

Verification infrastructure is temporarily impaired but no authoritative violation has been established.

The adapter follows the deployment-specific continuity policy.

UNSYNCD

The authoritative synchronization condition is false.

New Haven-mediated translations are blocked.

CIRCUIT_OPEN

The enforcement boundary is actively preventing protocol traffic until the required recovery conditions are satisfied.

4.38 RECOVERY

Recovery shall require fresh authoritative verification.

The adapter must not reopen the circuit merely because:

TimerExpired
TimerExpired

or:

ClientRequestedRestart
ClientRequestedRestart

Instead:

$\text{Recovery} \Rightarrow \text{FreshVerification} \wedge \text{PolicyValidation} \wedge \text{StateConsistencyRecovery} \setminus \text{Rightarrow} \text{FreshVerification} \setminus \text{and} \text{PolicyValidation} \setminus \text{and} \text{StateConsistency}$

Where necessary, queued events undergo reconciliation before downstream delivery.

4.39 NO-DESTRUCTIVE-FAILURE RULE

A synchronization failure must not destroy local clinical data.

Therefore:

$\text{CIRCUIT_OPEN} \Rightarrow \text{DELETE}(\text{LocalData}) \text{CIRCUIT_OPEN} \setminus \text{not} \setminus \text{Rightarrow} \text{DELETE}(\text{LocalData})$

and:

$\text{CIRCUIT_OPEN} \Rightarrow \text{ROLLBACK}(\text{ClinicalRecord}) \text{CIRCUIT_OPEN} \setminus \text{not} \setminus \text{Rightarrow} \text{ROLLBACK}(\text{ClinicalRecord})$

The circuit breaker applies only to the Haven-controlled interoperability boundary.

Local clinical operations remain institutionally governed.

4.40 SECURITY ASSUMPTION

The payment enclosure provides strongest protection against:

$\text{ApplicationLevelBypass} \text{ApplicationLevelBypass}$

It does not mathematically prevent:

$\text{PhysicalCompromise} \text{PhysicalCompromise}$
 $\text{PrivilegedHostCompromise} \text{PrivilegedHostCompromise}$

or:

$\text{CompleteInstitutionalInfrastructureCompromise} \text{CompleteInstitutionalInfrastructureCompromise}$

The protocol must therefore explicitly distinguish:

$\text{ProtocolSecurity} \text{ProtocolSecurity}$

from:

EndpointSecurityEndpointSecurity

A secure ledger contract cannot compensate for an unrestricted administrator who controls every component surrounding the adapter.

The architecture instead ensures that compromise of surrounding software does not automatically produce valid cryptographic settlement evidence.

4.41 FORMAL ZTPREE PROPERTY

For a billable event EE:

$$\text{Authorized}(E) = \text{Settlement}(E) \wedge \text{Identity}(E) \wedge \text{Integrity}(E) \wedge \text{Ordering}(E) \wedge \text{Policy}(E)$$
$$\text{Authorized}(E) = \text{Settlement}(E) \wedge \text{Identity}(E) \wedge \text{Integrity}(E) \wedge \text{Ordering}(E) \wedge \text{Policy}(E)$$

Therefore:

$$\neg \text{Settlement}(E) \Rightarrow \neg \text{Authorized}(E) \quad \neg \text{Settlement}(E) \Rightarrow \neg \text{Authorized}(E)$$

and consequently:

$$\neg \text{Authorized}(E) \Rightarrow \neg \text{CrossNetworkTranslation}(E) \quad \neg \text{Authorized}(E) \Rightarrow \neg \text{CrossNetworkTranslation}(E)$$

Hence:

$$\neg \text{Settlement}(E) \Rightarrow \neg \text{CrossNetworkTranslation}(E) \quad \neg \text{Settlement}(E) \Rightarrow \neg \text{CrossNetworkTranslation}(E)$$

This is the primary formal security property of the payment enclosure.

4.42 SECTION 4 COMPLETION

The Zero-Trust Payment Routing Enclosure Engine establishes the economic gate upon which Haven interoperability depends.

Its architectural model is:

Client $\rightarrow$ RequestClient $\rightarrow$ Request Contract $\rightarrow$ SettlementTruthContract $\rightarrow$ SettlementTruth Adapter $\rightarrow$ VerificationAdapter $\rightarrow$ Verification Protocol $\rightarrow$ AuthorizationProtocol $\rightarrow$ Authorization Network $\rightarrow$ TranslationNetwork $\rightarrow$ Translation

The client may initiate the transaction.

The client cannot declare it valid.

The protocol determines eligibility from independently verifiable evidence.

POLYGON / EVM SETTLEMENT ARCHITECTURE

5.1 Settlement Network

Polygon is designated as the initial EVM settlement environment for Haven's protocol-economic layer.

The settlement architecture shall treat the EVM ledger as an external source of independently verifiable transaction state.

The canonical relationship is:

Checkout→SettlementTransaction→Polygon→ContractState→ProtocolVerificationCheckout
\\rightarrow SettlementTransaction \\rightarrow Polygon \\rightarrow ContractState \\rightarrow ProtocolVerification

The client application is therefore not the final authority over payment status.

5.2 Contract Roles

The production settlement subsystem contains four logically distinct roles:

A. SETTLEMENT CONTRACT

Receives the designated protocol charge and executes deterministic accounting.

B. TREASURY DESTINATION

Receives the designated corporate allocation.

C. HAVEN VAULT

Receives the designated Haven protocol allocation.

D. VERIFICATION ADAPTER

Reads authoritative blockchain state and determines whether the associated event satisfies settlement requirements.

The adapter has read/verification authority.

It must not possess treasury withdrawal authority.

5.3 IMMUTABLE SPLITTER MODEL

The production splitter is defined as:

CSC_S

where:

CS=ImmutableSettlementContractC_S = ImmutableSettlementContract

The production contract shall not employ a conventional upgradeable proxy if immutability is a hard protocol requirement.

The expected contract identity consists of:

IC={ChainID,Address,CodeHash}I_C= \{ ChainID, Address, CodeHash \}

The adapter accepts settlement evidence only where:

Observed(IC)=Expected(IC)Observed(I_C)=Expected(I_C)

5.4 CONTRACT EXECUTION MODEL

A structured settlement operation is conceptually:

settle(ParticipantID,EventID,Nonce)settle(ParticipantID, EventID, Nonce)

with the corresponding value transfer:

msg.value=Fmsg.value=F

The contract validates:

ParticipantParticipant EventIDEventID NonceNonce AmountAmount

before executing allocation.

The resulting state transition is:

SC(t)→SettlementSC(t+1)S_C(t) \xrightarrow{\text{Settlement}} S_C(t+1)

where SCS_C represents contract-level protocol state.

5.5 ALLOCATION FUNCTION

For a percentage split:

$$FT = \lfloor F(1-r) \rfloor \quad F_T = \lceil F(1-r) \rceil \quad FV = F - FT \quad F_V = F - F_T$$

For:

$$r = 0.05$$

the resulting allocation is:

$$FT = \lfloor 0.95F \rfloor \quad F_T = \lceil 0.95F \rceil \quad FV = F - FT \quad F_V = F - F_T$$

The second calculation deliberately uses the remainder rather than independently rounding both values.

Therefore:

$$FT + FV = F \quad F_T + F_V = F$$

This creates a strict conservation invariant.

5.6 DESTINATION IMMUTABILITY

The production splitter must not permit an arbitrary caller to supply:

```
treasuryAddress  
vaultAddress
```

as mutable transaction parameters.

Instead:

$$DT = \text{ContractBoundDestination} \quad D_T = \text{ContractBoundDestination}$$

and:

$$DV = \text{ContractBoundDestination} \quad D_V = \text{ContractBoundDestination}$$

The destination configuration must be established during deployment and protected according to the protocol's immutability policy.

If governance-controlled destinations are later introduced, that constitutes a different security model and must be explicitly documented as such.

5.7 SETTLEMENT EVENT

Every successful settlement should emit an event equivalent in semantic function to:

SettlementRecorded(ParticipantID,EventID,Amount,TreasuryAmount,VaultAmount,Nonce)SettlementRecorded(ParticipantID, EventID, Amount, TreasuryAmount, VaultAmount, Nonce)

The event becomes an indexed forensic reference.

The adapter can subsequently reconstruct:

EventID→TransactionHash→Block→SettlementEvent
EventID \rightarrow TransactionHash \rightarrow Block \rightarrow SettlementEvent

without accessing the customer's internal database.

5.8 ON-CHAIN ACCOUNTING

The contract must maintain sufficient state to prevent:

- duplicate settlement;
- nonce reuse;
- unauthorized synchronization transitions;
- destination substitution;
- amount manipulation.

A conceptual state structure is:

StateC={UsedNonce,SettlementHash,ParticipantState,SynchronizationState}
State_C= \{ UsedNonce, SettlementHash, ParticipantState, SynchronizationState \}

The exact implementation may optimize storage, but the security properties must remain invariant.

5.9 TRANSACTION FINALITY

The adapter must distinguish:

Pending Pending Included Included Confirmed Confirmed Final Final

A transaction hash alone is insufficient.

The production policy must define the minimum finality condition:

Finality(Transaction)=TRUE
Finality(Transaction)=TRUE

before the settlement is treated as authoritative.

The required threshold shall account for the selected Polygon execution environment, operational risk tolerance, and the consequences of an incorrect authorization decision.

5.10 SETTLEMENT VERIFICATION

The adapter verifies:

$$\text{VerifySettlement}(E) = \text{ChainValid} \wedge \text{ContractValid} \wedge \text{TransactionValid} \wedge \text{EventValid} \wedge \text{AmountValid} \wedge \text{NonceValid} \wedge \text{FinalityValid}$$
$$\text{Verify}_{\{\text{settlement}\}}(E) = \text{ChainValid} \wedge \text{ContractValid} \wedge \text{TransactionValid} \wedge \text{EventValid} \wedge \text{AmountValid} \wedge \text{NonceValid} \wedge \text{FinalityValid}$$

Only when:

$$\text{VerifySettlement}(E) = 1 \wedge \text{Verify}_{\{\text{settlement}\}}(E) = 1$$

may the settlement predicate contribute to Haven event authorization.

5.11 CONTRACT BALANCE POLICY

The preferred settlement implementation minimizes idle assets held by the splitter.

Conceptually:

$$\text{Receive} \rightarrow \text{Split} \rightarrow \text{Forward} \quad \text{Receive} \rightarrow \text{Split} \rightarrow \text{Forward}$$

rather than:

$$\text{Receive} \rightarrow \text{Accumulate} \rightarrow \text{ManualWithdrawal} \quad \text{Receive} \rightarrow \text{Accumulate} \rightarrow \text{ManualWithdrawal}$$

where technically and economically practical.

This reduces the amount of value exposed to the splitter contract at any given time.

5.12 GAS AND EXECUTION COST

The protocol must account for the fact that EVM execution requires transaction fees.

Therefore:

$$\text{CustomerFee} \neq \text{GasCost} \quad \text{CustomerFee} \neq \text{GasCost}$$

unless the commercial model explicitly defines them as the same economic charge.

Gas sponsorship, relaying, batching, or account-abstraction mechanisms may be introduced independently.

The protocol tax must not become economically ambiguous merely because the underlying settlement transaction has variable execution costs.

POLYGON / EVM SETTLEMENT ARCHITECTURE

5.13 NATIVE ASSET AND TOKEN SETTLEMENT

The splitter architecture must explicitly define the settlement asset.

Three models are possible:

NativeAssetNativeAsset ERC20ERC20

or:

PaymentProcessor→OnChainSettlementAssetPaymentProcessor \rightarrow
OnChainSettlementAsset

The selected model must be established before production deployment because `receive()/fallback()` semantics, token transfer mechanics, decimal precision, approval requirements, and accounting behavior differ materially.

For ERC-20 settlement, the contract should use established safe-transfer patterns and must not assume that every token implements transfer behavior identically.

5.14 TOKEN ACCOUNTING

For an ERC-20 settlement asset:

BalancecontractBalance_{contract}

must be reconciled against:

SettlementLedgerSettlementLedger

The accounting invariant is:

$$\sum F_{received} = \sum F_{allocated} + F_{authorizedResidual}$$
$$\sum F_{received} = \sum F_{allocated} + F_{authorizedResidual}$$

where any residual must be explicitly defined.

Unexpected residual balances constitute an accounting exception requiring investigation.

5.15 EVENT CORRELATION

The settlement transaction must establish a deterministic relationship between:

EventIDEventID

and:

TransactionHashTransactionHash

A preferred model is to include a protocol event reference within the structured transaction call.

Conceptually:

$$\text{EventID} = H(\text{ParticipantID} \parallel \text{LocalEventID} \parallel \text{Nonce})$$
$$\text{EventID} = H(\text{ParticipantID} \parallel \text{LocalEventID} \parallel \text{Nonce})$$

The contract emits the resulting reference as part of the settlement event.

This enables:

$$\text{LocalEvent} \leftrightarrow \text{HavenEvent} \leftrightarrow \text{PolygonTransaction}$$
$$\text{LocalEvent} \leftrightarrow \text{HavenEvent} \leftrightarrow \text{PolygonTransaction}$$

without transferring the clinical payload onto the ledger.

5.16 SETTLEMENT REPLAY DEFENSE

The contract shall maintain an acceptance boundary:

$$\text{Accepted(EID)} = \text{TRUE}$$
$$\text{Accepted(EID)} = \text{TRUE}$$

after successful processing.

Any subsequent transaction attempting to reuse the same unique event reference must fail:

$$\text{Accepted(EID)} = \text{TRUE} \Rightarrow \text{NewSettlement(EID)} = \text{REJECT}$$
$$\text{Accepted(EID)} = \text{TRUE} \Rightarrow \text{NewSettlement(EID)} = \text{REJECT}$$

Nonce-level protection provides a second control:

$$\text{UsedNonce(P,N)} = \text{TRUE} \Rightarrow \text{Reuse(P,N)} = \text{REJECT}$$
$$\text{UsedNonce(P,N)} = \text{TRUE} \Rightarrow \text{Reuse(P,N)} = \text{REJECT}$$

The dual-control approach reduces the probability that a single identifier failure produces a financial duplication.

5.17 SYNCHRONIZATION STATE STORAGE

The contract may maintain:

$\text{mapping}(\text{address} \Rightarrow \text{bool})$

representing participant synchronization eligibility.

However, this Boolean must not be interpreted as a complete representation of protocol health.

A more rigorous authorization predicate is:

$\text{SyncValid}(P) = \text{SyncFlag}(P) \wedge \text{ContractIntegrity} \wedge \text{SettlementValidity} \wedge \text{PolicyValidity}$
 $\text{SyncValid}(P) = \text{SyncFlag}(P) \wedge \text{ContractIntegrity} \wedge \text{SettlementValidity} \wedge \text{PolicyValidity}$

The adapter therefore verifies the complete predicate rather than blindly trusting a single Boolean.

5.18 SYNCHRONIZATION STATE AUTHORITY

The contract must define precisely which authority can transition:

$\text{FALSE} \rightarrow \text{TRUE}$

or:

$\text{TRUE} \rightarrow \text{FALSE}$

Possible authority models include:

SettlementDerived
 $\text{SettlementDerived} \wedge \text{GovernanceDerived}$
 $\text{AttestationDerived}$

or a controlled combination.

The production architecture should avoid a design in which an ordinary client wallet can arbitrarily set its own synchronization flag.

Otherwise:

$\text{Client} \rightarrow \text{TRUE}$

would defeat the intended zero-trust model.

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

5.19 SETTLEMENT-DERIVED SYNCHRONIZATION

Where synchronization is directly derived from settlement, the state transition can be represented as:

$$\text{SyncP}(t+1) = \text{SettlementValid}(P, t) \quad \text{Sync_P}(t+1) = \text{SettlementValid}(P, t)$$

However, this creates an important design constraint:

A single historical payment cannot necessarily prove that a network remains continuously synchronized.

Therefore, the protocol must define whether synchronization means:

EverPaid EverPaid CurrentlyPaid CurrentlyPaid RecentlySettled RecentlySettled

or:

EligibleUnderCurrentPolicy EligibleUnderCurrentPolicy

Haven shall use the final interpretation:

$$\text{Sync} = \text{EligibleUnderCurrentPolicy} \quad \boxed{\text{Sync} = \text{EligibleUnderCurrentPolicy}}$$

rather than treating synchronization as a permanent payment receipt.

5.20 TEMPORAL AUTHORIZATION

For a synchronization lease of duration Δt :

$$\text{SyncP}(t) = 1 \quad \text{Sync_P}(t) = 1$$

only where:

$$t \leq t_{\text{lastValidSettlement}} + \Delta t \leq t_{\text{lastValidSettlement}} + \Delta t$$

A fresh settlement can extend the authorization window.

This creates:

$$\text{Settlement} \rightarrow \text{AuthorizationLease} \rightarrow \text{Synchronization} \quad \text{Settlement} \rightarrow \text{AuthorizationLease} \rightarrow \text{Synchronization}$$

rather than requiring the adapter to interpret one historical transaction as permanent authorization.

5.21 BLOCK REFERENCE

Every verification operation should bind its conclusion to a specific blockchain state.

Let:

B_k

represent the verified block.

Then:

$State(P, B_k)$

becomes the state against which the adapter makes its decision.

The adapter must record:

$BlockNumber$

and, where operationally appropriate:

$BlockHash$

alongside the verification result.

This prevents ambiguous statements such as “the wallet was synchronized” without identifying the ledger state from which that conclusion was derived.

5.22 FORENSIC RECONSTRUCTION

A complete settlement trail must permit reconstruction of:

$Participant \rightarrow EventID \rightarrow SettlementTransaction \rightarrow ContractBlock \rightarrow Allocation \rightarrow SynchronizationDecision$

The resulting forensic chain is:

$EID \rightarrow SettlementTx \rightarrow Block \rightarrow ContractEvent \rightarrow AdapterVerification \rightarrow CrossNetworkDecision$

This is the minimum evidence chain required to investigate a disputed protocol event.

5.23 CONTRACT FAILURE MODES

The settlement subsystem shall explicitly account for:

- reverted transactions;
- insufficient gas;
- incorrect asset;
- incorrect contract address;
- incorrect chain;
- nonce collision;
- duplicate event reference;
- invalid participant;
- unauthorized state mutation;
- RPC inconsistency;
- blockchain reorganization;
- destination transfer failure.

Each failure must produce a deterministic protocol outcome.

The system must never infer:

Unknown=SuccessUnknown = Success

5.24 SECTION 5 DESIGN PRINCIPLE

The Polygon layer is not the clinical system of record.

It is the **economic settlement and cryptographic attestation substrate**.

Its authoritative role is limited to the state explicitly committed to the ledger.

Therefore:

$\text{PolygonState} \neq \text{ClinicalState}$

but:

$\text{PolygonState} \rightarrow \text{ProofOfProtocolCondition}$

This separation preserves Haven's stateless interoperability model while giving the economic enforcement layer an independently verifiable foundation.

POLYGON / EVM SETTLEMENT ARCHITECTURE

5.25 CONTRACT-LEVEL SECURITY MODEL

The production settlement contract shall be designed under the assumption that every externally supplied transaction parameter is hostile until validated.

The contract shall independently validate:

CallerCaller ParticipantParticipant EventIDEventID NonceNonce AmountAmount AssetAsset
DestinationDestination

and applicable protocol state.

Client software shall never be treated as an authorization oracle.

5.26 REENTRANCY CONTROL

Where settlement execution transfers value to external contracts, the implementation shall account for reentrant execution.

The preferred execution sequence is:

Validate→UpdateState→Transfer→EmitEventValidate \rightarrow UpdateState \rightarrow
Transfer \rightarrow EmitEvent

rather than:

Transfer→UpdateStateTransfer \rightarrow UpdateState

The contract shall employ an appropriate reentrancy protection pattern where external calls create exposure.

The security invariant is:

ReentrantCall⇒DuplicateSettlementReentrantCall \not\rightarrow DuplicateSettlement

5.27 CHECKS-EFFECTS-INTERACTIONS

Settlement execution shall follow:

Checks→Effects→InteractionsChecks \rightarrow Effects \rightarrow Interactions

All critical accounting state must be committed before value is transferred externally.

For example:

$$\text{UsedNonce}[A][N] \leftarrow \text{TRUE} \text{UsedNonce}[A][N] \rightarrow \text{TRUE}$$

must occur before the contract performs an external value transfer.

This prevents an external call from attempting to reuse the same settlement state during execution.

5.28 ACCESS CONTROL

Administrative functions, where unavoidable, shall be minimized.

If an administrative operation exists, the permission boundary must be explicit:

$$\text{Function} \rightarrow \text{AuthorizedRole} \rightarrow \text{AuthenticatedCaller} \text{Function} \rightarrow \text{AuthorizedRole} \rightarrow \text{AuthenticatedCaller}$$

The following authorities must remain distinct:

TreasuryAuthority TreasuryAuthority ProtocolGovernanceProtocolGovernance
ParticipantAuthority ParticipantAuthority AdapterIdentityAdapterIdentity

No ordinary participant should receive contract-wide administrative privileges.

5.29 ZERO-TRUST CONTRACT INTERFACE

The contract interface must assume that:

`msg.sender`

may be malicious.

Likewise:

`msg.value`

must be validated against the expected economic condition.

Transaction call data must be treated as untrusted input.

Therefore:

$$\text{UnvalidatedInput} \rightarrow \text{StateMutation} = \text{FALSE} \text{UnvalidatedInput} \rightarrow \text{StateMutation} = \text{FALSE}$$

This applies even where the caller is an officially registered participant.

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

Registration establishes identity.

It does not establish perpetual trust.

5.30 CONTRACT DEPLOYMENT ARTIFACT

Before production deployment, the exact executable artifact shall be deterministically identified.

The deployment record shall contain:

ArtifactHashArtifactHash CompilerVersionCompilerVersion CompilerSettingsCompilerSettings
SourceCommitSourceCommit DeploymentAddressDeploymentAddress ChainIDChainID
DeploymentBlockDeploymentBlock

The production adapter shall pin the expected contract identity against this deployment record.

5.31 INDEPENDENT VERIFICATION

Production contract verification shall be performed independently of the development environment.

At minimum:

SourceCode $\leftrightarrow$ CompiledBytecode $\leftrightarrow$ DeployedBytecodeSourceCode $\rightarrow$
CompiledBytecode $\rightarrow$ DeployedBytecode

must be reconciled.

The deployment address alone is insufficient evidence of contract integrity.

5.32 AUDIT REQUIREMENT

Before production economic activation, the settlement contract should undergo:

1. static analysis;
2. unit testing;
3. invariant testing;
4. fuzz testing;
5. adversarial testing;
6. independent code review;
7. deployment verification;
8. controlled testnet execution.

For high-value production infrastructure, independent smart-contract security review is mandatory as a release gate.

5.33 EMERGENCY MODEL

An immutable contract intentionally limits emergency intervention.

This produces a deliberate trade-off:

Immutability→ReducedUpgradeRiskImmutability \rightarrow ReducedUpgradeRisk

but:

Immutability→ReducedEmergencyControlImmutability \rightarrow ReducedEmergencyControl

Haven must therefore prefer prevention, monitoring, and operational isolation over an administrator-controlled upgrade mechanism.

If an emergency stop is deemed necessary, its existence must be treated as an explicit deviation from absolute immutability and documented as part of the protocol's trust model.

5.34 SETTLEMENT CONTRACT RELEASE GATE

The contract shall not be considered production-ready until:

SecurityTests=PASSSecurityTests=PASS

AccountingInvariants=PASSAccountingInvariants=PASS

DeploymentVerification=PASSDeploymentVerification=PASS

DestinationVerification=PASSDestinationVerification=PASS

ReplayTests=PASSReplayTests=PASS FailureTests=PASSFailureTests=PASS

and:

IndependentReview=PASSIndependentReview=PASS

The economic rail is a critical infrastructure component and must therefore have a stricter release threshold than ordinary application software.

5.35 SECTION 5 CLOSURE

The Polygon/EVM settlement layer establishes the ledger-backed economic boundary for Haven.

Its core guarantees are:

DeterministicAllocationDeterministicAllocation EventCorrelationEventCorrelation

ReplayResistanceReplayResistance ContractIntegrityContractIntegrity

SettlementVerifiabilitySettlementVerifiability TreasuryIsolationTreasuryIsolation

These properties establish the settlement substrate upon which the multi-chain coordination layer can operate.

MULTI-CHAIN ABSTRACTION ROUTING LAYER

6.1 Architectural Objective

Haven shall not make its core protocol state machine dependent upon a single cross-chain messaging provider.

The protocol therefore separates:

ApplicationStateApplicationState

from:

TransportMechanismTransportMechanism

The routing architecture is:

HavenEvent→RoutingPolicy→TransportAdapter→InteroperabilityNetwork→DestinationHaven
Event \rightarrow RoutingPolicy \rightarrow TransportAdapter \rightarrow
InteroperabilityNetwork \rightarrow Destination

This creates provider abstraction at the protocol boundary.

6.2 TRANSPORT-AGNOSTIC MESSAGE MODEL

A Haven cross-chain message shall possess a provider-independent representation:

$MH = \{MessageID, SourceChain, DestinationChain, SourceDomain, DestinationDomain, EventCommitment, Nonce, Policy, Expiry, Signature\}$
 $M_H = \{ MessageID, SourceChain, DestinationChain, SourceDomain, DestinationDomain, EventCommitment, Nonce, Policy, Expiry, Signature \}$

The transport adapter converts this object into the message format required by the selected interoperability network.

Therefore:

$MH \neq M_{LayerZero} M_H \neq M_{\{LayerZero\}}$

and:

$MH \neq M_{CCIP} M_H \neq M_{\{CCIP\}}$

The Haven message remains logically identical regardless of transport provider.

6.3 ROUTING DECISION FUNCTION

The routing engine evaluates:

$Route = f(\text{Source}, \text{Destination}, \text{Asset}, \text{MessageType}, \text{SecurityPolicy}, \text{Availability}, \text{Cost}, \text{Finality})$
 $Route = f(\text{Source}, \text{Destination}, \text{Asset}, \text{MessageType}, \text{SecurityPolicy}, \text{Availability}, \text{Cost}, \text{Finality})$

The result identifies an approved transport path.

Conceptually:

$Route(MH) = \{\text{Provider}, \text{Path}, \text{SecurityProfile}\}$ $Route(M_H) = \{\text{Provider}, \text{Path}, \text{SecurityProfile}\}$

A provider must not be selected merely because it is available.

The selected route must satisfy the security policy attached to the message class.

6.4 PROVIDER ABSTRACTION

The initial transport abstraction supports two principal provider families:

LayerZeroLayerZero

and:

Chainlink CCIPChainlink\ CCIP

These are treated as transport infrastructure rather than as owners of Haven state.

The protocol therefore avoids:

$Haven = LayerZeroHaven = LayerZero$

and:

$Haven = CCIPHaven = CCIP$

The correct relationship is:

$Haven \supset TransportAdapters \supset InteroperabilityProviders$ $Haven \supset TransportAdapters \supset InteroperabilityProviders$

6.5 LAYERZERO ROUTING ADAPTER

The LayerZero adapter shall translate Haven messages into the LayerZero messaging model.

LayerZero V2 uses endpoint contracts on supported chains and separates messaging from verification and execution through its protocol architecture. Its security model can incorporate configurable Decentralized Verifier Networks (DVNs), while execution is handled through Executors.

The Haven adapter shall therefore treat the LayerZero endpoint as a transport boundary:

HavenAdapter→LayerZeroEndpoint→DestinationEndpoint
HavenAdapter \rightarrow LayerZeroEndpoint \rightarrow DestinationEndpoint

The Haven protocol remains responsible for determining whether the message is valid.

LayerZero remains responsible for transporting and verifying the message according to the configured LayerZero security configuration.

6.6 LAYERZERO SECURITY PARAMETERS

For each LayerZero route, Haven shall explicitly define:

SecurityProfileLZ={Endpoint,MessageLib,DVNSet,DVNThreshold,Executor,Options,Destination,Version}
SecurityProfile_{LZ} = \{ Endpoint, MessageLib, DVNSet, DVNThreshold, Executor, Options, Destination, Version \}

The exact DVN set and threshold must be selected according to the risk classification of the Haven message.

High-criticality clinical coordination messages should not use an unreviewed or minimally secured transport configuration.

6.7 LAYERZERO NONCE DOMAIN

LayerZero channels maintain ordered message sequencing through nonces.

Haven shall maintain its own protocol-level event and message identifiers independently.

Therefore:

NonceHaven≠NonceLZ
Nonce_{Haven} \neq Nonce_{LZ}

but the two may be correlated:

$\text{HavenMessageID} \leftrightarrow \text{LayerZeroNonce} \text{HavenMessageID} \rightarrow \text{LayerZeroNonce}$

This prevents transport-specific sequencing from becoming the sole source of Haven event ordering.

6.8 CROSS-CHAIN MESSAGE ACCEPTANCE

A destination adapter shall accept a transported Haven event only after verifying:

$\text{MessageAuthenticity} \wedge \text{SourceDomain} \wedge \text{DestinationDomain} \wedge \text{HavenEventIntegrity} \wedge \text{Policy} \wedge \text{ReplayProtection}$
 $\text{MessageAuthenticity} \wedge \text{SourceDomain} \wedge \text{DestinationDomain} \wedge \text{HavenEventIntegrity} \wedge \text{Policy} \wedge \text{ReplayProtection}$

Transport success alone is insufficient.

Therefore:

$\text{DeliveredMessage} \Rightarrow \text{AuthorizedHavenEvent} \text{DeliveredMessage} \not\Rightarrow \text{AuthorizedHavenEvent}$

The destination must perform protocol-level validation after transport delivery.

6.9 CHAINLINK CCIP ADAPTER

Chainlink CCIP shall be represented as a second transport implementation beneath the Haven routing abstraction.

The logical architecture is:

$\text{HavenEvent} \rightarrow \text{CCIPAdapter} \rightarrow \text{CCIP} \rightarrow \text{DestinationChain} \text{HavenEvent} \rightarrow \text{CCIPAdapter} \rightarrow \text{CCIP} \rightarrow \text{DestinationChain}$

CCIP's role is transport and cross-chain message delivery under its configured security model.

Haven retains ownership of its protocol semantics.

The CCIP adapter must translate:

$\text{MH} \rightarrow \text{MCCIPM_H} \rightarrow \text{M}_{\{\text{CCIP}\}}$

without changing the semantic meaning of the Haven event.

6.10 ROUTE EQUIVALENCE

If two approved transport providers deliver the same Haven event:

$MH \rightarrow LZDM_H \xrightarrow{\{LZ\}} D$

and:

$MH \rightarrow CCIPDM_H \xrightarrow{\{CCIP\}} D$

the destination protocol state must converge on the same logical result:

$StateDLZ = StateDCCIPState_D^{\{LZ\}} = State_D^{\{CCIP\}}$

provided both messages satisfy the same Haven authorization predicates.

Transport-provider variation must not produce semantic variation in the Haven state machine.

6.11 PROVIDER FAILURE

If a transport provider becomes unavailable:

$ProviderA = UNAVAILABLE \rightarrow Provider_A = UNAVAILABLE$

the routing layer may select an approved alternative:

$ProviderB \rightarrow Provider_B$

subject to:

$SecurityProfileB \geq RequiredSecurityProfile \rightarrow SecurityProfile_B \geq RequiredSecurityProfile$

A cheaper or faster provider must not automatically replace a more secure provider when the event's policy requires the higher security threshold.

6.12 ROUTING PRINCIPLE

The multi-chain layer therefore establishes:

$One\ Protocol \neq One\ Transport \rightarrow \boxed{One\ Protocol \neq One\ Transport}$

Haven owns the event semantics, authorization model, state commitments, and routing policy.

Interoperability providers supply transport infrastructure beneath that abstraction.

This separation is mandatory for long-term protocol portability.

MULTI-CHAIN ABSTRACTION ROUTING LAYER

6.13 CHAINLINK CCIP SECURITY PROFILE

For each CCIP route, Haven shall maintain a route-specific security profile:

$SecurityProfile_{CCIP} = \{Router, SourceChain, DestinationChain, ChainSelector, Receiver, MessageType, FinalityPolicy\}$
 $SecurityProfile_{\{CCIP\}} = \{Router, SourceChain, DestinationChain, ChainSelector, Receiver, MessageType, FinalityPolicy\}$

The exact production values shall be populated during deployment and network onboarding.

The routing abstraction must not hard-code assumptions about a single destination chain.

6.14 CHAIN SELECTOR SEPARATION

CCIP uses chain selectors to identify supported blockchain networks.

Haven shall therefore maintain two distinct identifiers:

$HavenChainID$ $HavenChainID$

and:

$CCIPChainSelector$ $CCIPChainSelector$

They must never be treated as interchangeable.

Conceptually:

$NetworkIdentity = \{EVMChainID, CCIPChainSelector, TransportDomain\}$
 $NetworkIdentity = \{EVMChainID, CCIPChainSelector, TransportDomain\}$

This prevents an interoperability provider's identifier from becoming the canonical identity of the Haven network.

6.15 CCIP ROUTER BINDING

The CCIP adapter shall interact only with an approved Router address for the relevant network.

Conceptually:

$ApprovedRouter(C) = R_C$

where CC represents the applicable chain.

A message submitted through an unrecognized Router shall not be treated as a valid Haven transport event.

The production configuration therefore requires:

ObservedRouter=ApprovedRouterObservedRouter=ApprovedRouter

before route activation.

6.16 CROSS-CHAIN RECEIVER VALIDATION

The destination receiver must validate the originating transport context.

At minimum:

SourceChainValidSourceChainValid SourceSenderValidSourceSenderValid
DestinationValidDestinationValid MessageIDValidMessageIDValid
ReplayStateValidReplayStateValid

must all evaluate successfully.

Therefore:

$$\text{Receive}(M) = \bigwedge_{k=1}^n \text{Predicate}_k \text{Receive}(M) = \bigwedge_{k=1}^n \text{Predicate}_k$$

If any mandatory predicate fails:

$$\text{Receive}(M) = \text{FALSE} \text{Receive}(M) = \text{FALSE}$$

6.17 ROUTE REGISTRY

Haven shall maintain a logical route registry:

$$R = \{r_1, r_2, \dots, r_n\} R = \{r_1, r_2, \dots, r_n\}$$

Each route r_i contains:

$$r_i = \{\text{Source}, \text{Destination}, \text{Provider}, \text{SecurityProfile}, \text{FinalityRequirement}, \text{Status}\} r_i = \{\text{Source}, \text{Destination}, \text{Provider}, \text{SecurityProfile}, \text{FinalityRequirement}, \text{Status}\}$$

Example:

```
Route ID: HVN-LZ-001
Source: Polygon
Destination: [DESTINATION_CHAIN]
Provider: LayerZero
Security Profile: [DVN_CONFIGURATION]
Status: APPROVED
```

and:

```
Route ID: HVN-CCIP-001
Source: Polygon
Destination: [DESTINATION_CHAIN]
Provider: Chainlink CCIP
Security Profile: [CCIP_CONFIGURATION]
Status: APPROVED
```

These are configuration records, not clinical data.

6.18 ROUTE AUTHORIZATION

A route must satisfy:

$\text{RouteApproved} \wedge \text{ProviderApproved} \wedge \text{DestinationApproved} \wedge \text{SecurityProfileValid} \wedge \text{RouteApproved}$
 $\wedge \text{ProviderApproved} \wedge \text{DestinationApproved} \wedge \text{SecurityProfileValid}$

before it can carry Haven events.

Consequently:

$\text{UnknownRoute} \rightarrow \text{REJECTUnknownRoute} \rightarrow \text{REJECT}$

The protocol shall never dynamically select an arbitrary cross-chain transport provider merely because an endpoint is reachable.

6.19 CROSS-CHAIN EVENT COMMITMENT

Before transmission, the source adapter calculates:

$$C_E = H(\text{EventID} \parallel \text{SourceDomain} \parallel \text{DestinationDomain} \parallel \text{PayloadCommitment} \parallel \text{Nonce})$$
$$C_E = H(\text{EventID} \parallel \text{SourceDomain} \parallel \text{DestinationDomain} \parallel \text{PayloadCommitment} \parallel \text{Nonce})$$

The destination independently reconstructs the expected commitment.

Acceptance requires:

$$C_{received} = CEC_{\{received\}} = C_E$$

This prevents transport-layer delivery from being confused with event-level integrity.

6.20 CROSS-CHAIN REPLAY PROTECTION

Every Haven message shall have a unique protocol identifier:

$$MID_{MID}$$

and destination-domain nonce:

$$NDN_D$$

The destination maintains sufficient state to establish:

$$Consumed(MID) = TRUE \wedge Consumed(MID) = TRUE$$

after successful application.

A replayed message therefore satisfies:

$$Consumed(MID) = TRUE \Rightarrow REJECT \wedge Consumed(MID) = TRUE \rightarrow REJECT$$

This protection remains active even if the underlying transport provider independently supplies replay protection.

Haven shall not outsource its core protocol security assumptions entirely to a transport provider.

6.21 SOURCE AUTHORITY

The destination must verify that the message originated from an approved Haven source.

Conceptually:

$$SourceAuthority = Verify(SourceDomain, SourceApplication, SourceContract, MessageAuthenticator) \vee SourceAuthority = Verify(SourceDomain, SourceApplication, SourceContract, MessageAuthenticator)$$

The protocol shall not authorize a message solely because it originated from a valid blockchain transaction.

A valid transaction from an unauthorized application remains unauthorized.

6.22 CROSS-CHAIN STATE CONVERGENCE

For a valid event E :

$State_A \xrightarrow{E} State_B$

The destination must reach:

$State_B = F(State_A, E)$

according to the Haven state transition function.

If the same event arrives through another approved transport:

$F(State_A, E) = F(State_A, E)$

The transport provider must therefore remain semantically invisible to the application-level state machine.

6.23 PROVIDER FAILOVER

Provider failover is permitted only when the alternative route satisfies the event's security policy.

Define:

$Eligible(P) = Availability(P) \wedge Security(P) \wedge Policy(P) \wedge Destination(P)$

Then:

$Route_{Selected} = \arg\max_P Eligible(P)$

subject to the routing policy.

No failover mechanism may silently downgrade a critical message from a required security profile to a weaker one.

6.24 ROUTING OBSERVABILITY

Every cross-chain route shall generate operational telemetry containing, at minimum:

MessageID SourceDomain DestinationDomain
Provider SubmissionReference
DeliveryReference VerificationState
FinalityState

This metadata is operational evidence.

It is not a replacement for the underlying clinical system of record.

6.25 TRANSPORT INDEPENDENCE PRINCIPLE

The Haven protocol is considered transport-independent when:

$\text{StateTransition} \perp \text{StateTransition}$

remains unchanged despite replacement of:

$\text{LayerZero} \perp \text{LayerZero}$

with:

$\text{CCIP} \perp \text{CCIP}$

or another subsequently approved transport.

Therefore:

$\text{ProtocolSemantics} \perp \text{TransportImplementation} \perp \text{ProtocolSemantics} \perp \text{TransportImplementation}$

The protocol's long-term interoperability strategy is consequently based on an abstraction boundary rather than permanent dependence on a single messaging network.

CANONICAL INFRASTRUCTURE REGISTRY & INTEROPERABILITY SPECIFICATIONS

7.1 CANONICAL ADDRESS REGISTRY & EXECUTION INVARIANTS

The Haven Protocol shall operate under a single canonical Polygon settlement address for the initial production architecture.

7.1.1 CANONICAL POLYGON ADDRESS

```
0xC02E7Fe6AAfFF15F0bc4636B53E6d9D20898c4ac\boxed{
0xC02E7Fe6AAfFF15F0bc4636B53E6d9D20898c4ac }
```

This address shall constitute the sole designated Polygon USDC settlement and treasury destination for the Haven Protocol.

No secondary Haven treasury wallet, vault wallet, or alternative Polygon settlement destination shall be required by the baseline architecture.

7.1.2 SINGLE-ADDRESS INVARIANT

All Haven economic settlement references within the initial production configuration shall resolve to:

```
0xC02E7Fe6AAfFF15F0bc4636B53E6d9D20898c4ac
```

Therefore:

```
DestinationHaven=0xC02E7Fe6AAfFF15F0bc4636B53E6d9D20898c4acDestination_{Haven}
= 0xC02E7Fe6AAfFF15F0bc4636B53E6d9D20898c4ac
```

Any alternative destination presented by a client application or local checkout configuration shall be treated as unauthorized unless explicitly introduced through a future protocol revision.

7.1.3 ADDRESS NON-SUBSTITUTION

The client application shall not be permitted to replace the canonical Haven address with an alternative address through frontend configuration, checkout parameters, environment variables, or client-side verification flags.

The authoritative destination is determined by the production protocol configuration.

Therefore:

```
ObservedDestination≠CanonicalDestinationObservedDestination \neq CanonicalDestination
```

shall result in:

ERR_CANONICAL_DESTINATION_MISMATCH

7.1.4 SINGLE DESTINATION MODEL

The baseline Haven economic architecture is:

Customer $\rightarrow$ HavenSettlement $\rightarrow$ CanonicalPolygonAddress
Customer $\searrow$ HavenSettlement $\searrow$ CanonicalPolygonAddress

where:

CanonicalPolygonAddress = 0xC02E7Fe6AAFF15F0bc4636B53E6d9D20898c4ac
CanonicalPolygonAddress = 0xC02E7Fe6AAFF15F0bc4636B53E6d9D20898c4ac

The protocol shall not require separate treasury and vault destinations in its initial deployment configuration.

7.1.5 SMART-CONTRACT DEPLOYMENT DISTINCTION

The canonical Polygon address specified above shall not be represented in technical documentation as a smart-contract address unless on-chain verification establishes that the address contains the required deployed contract code.

If the address is an externally owned account (EOA), it shall remain classified as the canonical Polygon settlement/treasury wallet.

If a future deployment requires a smart-contract execution layer, that contract shall be technically connected to the canonical destination without falsely representing the destination wallet as the contract itself.

Accordingly:

WalletAddress $\neq$ ContractAddress

unless:

RuntimeCode(Address) $\neq$ 0

and the deployed code satisfies the Haven contract specification.

7.1.6 PRODUCTION VERIFICATION

Before activation, engineers shall independently verify:

Network=PolygonNetwork=Polygon ChainID=137ChainID=137
Address=CanonicalAddressAddress=CanonicalAddress Asset=USDCAsset=USDC

and the applicable contract or wallet classification.

The verification result shall be recorded in the production deployment manifest.

7.1.7 ADDRESS CHANGE POLICY

The canonical Polygon address shall remain fixed for the initial Haven deployment.

Any future change shall constitute a formal protocol configuration change and shall not be performed through client-side application configuration.

AddressChange⇒GovernanceControlledMigrationAddressChange \rightarrow
GovernanceControlledMigration

No informal replacement address shall become authoritative.

7.1.8 CANONICAL ADDRESS INVARIANT

The baseline production invariant is:

All Haven Polygon Settlement References→0xC02E7Fe6AAfFF15F0bc4636B53E6d9D20898c4ac\boxed{ All\ Haven\ Polygon\ Settlement\ References \rightarrow
0xC02E7Fe6AAfFF15F0bc4636B53E6d9D20898c4ac }

The single-address architecture eliminates unnecessary fragmentation between separate Haven treasury, vault, and settlement destinations while preserving the requirement that engineers distinguish an EOA from an executable smart contract.

7.2 CROSS-CHAIN TRANSPORT ABSTRACTION

Haven shall maintain a transport abstraction layer above individual interoperability providers.

The protocol semantic layer shall remain independent from the underlying transport implementation.

Conceptually:

HavenEvent→TransportAbstraction→LayerZero V2HavenEvent \rightarrow
TransportAbstraction \rightarrow LayerZero\ V2

or:

HavenEvent→TransportAbstraction→Chainlink CCIPHavenEvent \rightarrow
TransportAbstraction \rightarrow Chainlink\ CCIP

The transport provider carries the message.

It does not redefine the Haven event semantics.

7.2.1 TRANSPORT PROVIDER INDEPENDENCE

The protocol shall not permanently couple its event model to a single interoperability provider.

The canonical abstraction is:

Message=Payload+Source+Destination+Nonce+Commitment+ExecutionPolicyMessage =
Payload + Source + Destination + Nonce + Commitment + ExecutionPolicy

The transport implementation may vary while the protocol-level message semantics remain constant.

7.2.2 TRANSPORT SELECTION

A route may select LayerZero V2, Chainlink CCIP, or another subsequently authorized transport provider according to:

RoutePolicyRoutePolicy DestinationSupportDestinationSupport
SecurityRequirementsSecurityRequirements AvailabilityAvailability

and:

GovernanceAuthorizationGovernanceAuthorization

No client application may arbitrarily substitute an unapproved transport endpoint.

7.2.3 FINAL TRANSPORT INVARIANT

The protocol shall preserve the following invariant:

TransportFailure⇒ClinicalDataCorruption\boxed{ TransportFailure \not\rightarrow
ClinicalDataCorruption }

and:

$$\boxed{\text{TransportReplacement} \Rightarrow \text{ProtocolSemanticReplacement} \wedge \text{TransportReplacement} \not\Rightarrow \text{ProtocolSemanticReplacement}}$$

The transport layer remains an interchangeable infrastructure mechanism beneath the Haven coordination semantics.

EDGE INTEROPERABILITY ARCHITECTURE

8.1 EDGE ADAPTER ROLE

The Haven Edge Adapter is the execution boundary between a participating healthcare institution and the Haven coordination protocol.

It operates adjacent to existing infrastructure such as:

EMREMR LISLIS PharmacySystemPharmacySystem HMOClaimsSystemHMOClaimsSystem

and:

POS/CheckoutPOS/Checkout

The adapter does not replace these systems.

Its function is to translate authorized local state changes into Haven protocol events and to apply authorized Haven events to downstream local workflows.

8.2 LOCAL SYSTEM BOUNDARY

The local architecture is represented as:

ClinicalSystem→Adapter→HavenProtocolClinicalSystem \rightarrow Adapter \rightarrow HavenProtocol

The reverse path is:

HavenProtocol→Adapter→ClinicalSystemHavenProtocol \rightarrow Adapter \rightarrow ClinicalSystem

The adapter therefore forms a controlled boundary rather than becoming a new centralized clinical database.

8.3 ADAPTER TRUST POSITION

The adapter must not blindly trust the originating application.

Its trust model is:

Application→UntrustedInputApplication \rightarrow UntrustedInput
 Adapter→ValidationAdapter \rightarrow Validation Protocol→AuthorizationProtocol
 \rightarrow Authorization

The adapter verifies the protocol conditions independently before allowing a cross-network event to proceed.

8.4 EVENT INTERCEPTION

When a local system produces a qualifying event:

E_{localE_local}

the adapter extracts only the protocol-required metadata.

It generates:

E_{HavenE_Haven}

through deterministic normalization:

$E_{Haven} = N(E_{local})E_{_Haven} = N(E_{_local})$

where NN is the approved normalization function.

The complete clinical database is not required to pass through the adapter.

8.5 ASYNCHRONOUS QUEUE

Haven shall use asynchronous queues between local event production and protocol transmission.

Conceptually:

Producer→Queue→Validator→Settlement→Router
 Producer \rightarrow Queue \rightarrow
 Validator \rightarrow Settlement \rightarrow Router

This prevents temporary network unavailability from forcing the local clinical system to become synchronously dependent on the Haven network.

8.6 EVENT QUEUE STATES

Each event may transition through:

CREATED
 VALIDATING
 AWAITING_SETTLEMENT
 AWAITING_SETTLEMENT
 SETTLED
 SETTLED

QUEUEDQUEUED TRANSMITTEDTRANSMITTED DELIVEREDEDELIVERED
APPLIEDAPPLIED

or:

RECONCILIATION_REQUIREDRECONCILIATION_REQUIRED

No event may bypass mandatory states defined by its policy class.

8.7 PAYMENT GATE LOCATION

For billable events, the payment verification gate is positioned **before protocol-authorized cross-network transmission**.

The logical pipeline is:

LocalEvent→FeeDetermination→Settlement→OnChainVerification→ProtocolAuthorization→
CrossNetworkTransmissionLocalEvent \rightarrow FeeDetermination \rightarrow Settlement
\rightarrow OnChainVerification \rightarrow ProtocolAuthorization \rightarrow
CrossNetworkTransmission

Therefore, the adapter does not simply transmit first and ask whether the fee was paid later.

8.8 CLIENT APPLICATION LIMITATION

The client application may provide:

EventDataEventData

but cannot provide authoritative:

SettlementValidSettlementValid

or:

NetworkSynchronizedNetworkSynchronized

values.

These are independently determined by the protocol verification layer.

8.9 RPC VERIFICATION

At the configured interval:

$T_{poll}=60s$ $T_{\{poll\}}=60s$

the adapter queries the designated blockchain state.

The verification process evaluates:

ChainChain ContractContract CodeHashCodeHash SynchronizationStateSynchronizationState
and applicable settlement evidence.

The result is converted into an internal authorization state.

8.10 LOCAL CIRCUIT BREAKER

If the authoritative synchronization state is invalid:

$SyncValid=FALSE$ $SyncValid=FALSE$

the adapter transitions to:

CIRCUIT_OPENCIRCUIT_OPEN

New Haven-mediated cross-network translations are blocked.

Local clinical records remain untouched.

8.11 QUEUE PRESERVATION

Events blocked by the circuit breaker shall not automatically be destroyed.

Subject to data-retention and clinical-safety policy:

$BlockedEvent \rightarrow DurableQueue$ $BlockedEvent \rightarrow DurableQueue$

The event can subsequently undergo:

$Revalidation \rightarrow Reconciliation \rightarrow Transmission$ $Revalidation \rightarrow Reconciliation$
 $\rightarrow Transmission$

after authorization is restored.

8.12 IDEMPOTENT DELIVERY

The adapter must preserve the event identifier across retries:

$$EID_{retry} = EID_{original} EID_{\{retry\}} = EID_{\{original\}}$$

This permits downstream systems to identify repeated delivery attempts as the same logical protocol event.

8.13 LOCAL DATABASE NON-OWNERSHIP

The adapter shall not become a shadow centralized EMR.

Temporary operational buffers may exist, but the architecture must distinguish:

$$OperationalBuffer \neq ClinicalSystemOfRecord$$

The institution remains responsible for its clinical records.

Haven coordinates state transitions between institutions.

8.14 EDGE FAILURE ISOLATION

If the Haven edge process fails:

$$HavenEdge = DOWN$$

the local institution's clinical system must not automatically lose access to its own locally governed records.

The failure domain should remain bounded to:

$$InteroperabilityLayer$$

where operationally safe.

8.15 EDGE SECURITY PRINCIPLE

The Edge Adapter is therefore neither:

$$DatabaseOwner$$

nor:

$$ClinicalAuthority$$

nor:

TreasuryControllerTreasuryController

nor:

BlockchainAdministratorBlockchainAdministrator

It is an enforcement and translation component positioned at the interoperability boundary.

9.0 PROTOCOL GOVERNANCE AND SECURITY DOMAINS

9.1 GOVERNANCE OBJECTIVE

Haven governance shall establish enforceable boundaries between protocol operation, economic custody, client administration, and clinical-system ownership.

The governing principle is:

NoSingleActor→TotalProtocolControlNoSingleActor \rightarrow TotalProtocolControl

Authority shall instead be partitioned across independent security domains.

9.2 SECURITY DOMAIN MODEL

The production architecture contains five principal domains:

D1=ClinicalD_1=Clinical D2=EdgeD_2=Edge D3=ProtocolD_3=Protocol
D4=SettlementD_4=Settlement D5=TreasuryD_5=Treasury

Each domain possesses a defined authority envelope.

9.3 CLINICAL DOMAIN

The clinical domain contains:

EMREMR LISLIS PharmacyPharmacy HMOHMO

and other institutional systems.

Its authority is over the institution's own clinical records and workflows.

It does not possess authority over Haven's settlement contract.

9.4 EDGE DOMAIN

The Edge domain contains the Haven Adapter and associated local infrastructure.

Its authority is limited to:

NormalizeNormalize ValidateValidate QueueQueue VerifyVerify TransmitTransmit

The adapter must not possess:

TreasuryWithdrawalAuthorityTreasuryWithdrawalAuthority

or:

ContractUpgradeAuthorityContractUpgradeAuthority

9.5 PROTOCOL DOMAIN

The protocol domain defines:

EventSchemasEventSchemas StateTransitionsStateTransitions RoutingPoliciesRoutingPolicies
MessageValidityMessageValidity ReplayRulesReplayRules

and:

SynchronizationRulesSynchronizationRules

These rules constitute the Haven protocol state machine.

9.6 SETTLEMENT DOMAIN

The settlement domain contains the blockchain-based economic enforcement mechanism.

Its primary components are:

SplitterContractSplitterContract TreasuryDestinationTreasuryDestination
HavenVaultHavenVault

and:

SettlementVerificationSettlementVerification

The settlement domain is independently verifiable from client application infrastructure.

9.7 TREASURY DOMAIN

The treasury domain contains the Safe multisignature custody architecture.

The relationship is:

Splitter→Safe→AuthorizedSignersSplitter \rightarrow Safe \rightarrow AuthorizedSigners

The Edge Adapter has no private-key access.

The frontend has no private-key access.

The clinical institution has no unilateral withdrawal authority over Haven-controlled allocations.

9.8 GOVERNANCE SEPARATION

Governance authority shall be separated from operational execution.

A governance actor may define:

PolicyPolicy

without possessing:

TreasuryKeysTreasuryKeys

Likewise, a treasury signer may authorize a treasury transaction without possessing authority to alter clinical-system state.

This separation limits the blast radius of credential compromise.

9.9 CONTROL MATRIX

Domain	Primary Authority	Must Not Control
Clinical	Institution	Haven Treasury
Edge	Protocol Adapter	Private Keys
Protocol	Haven Governance	Clinical Records
Settlement	Smart Contract	Client Configuration
Treasury	Safe Signers	Clinical Workflow

The matrix is a structural security boundary.

9.10 ZERO-TRUST PRINCIPLE

Every interaction crossing a domain boundary shall be treated as untrusted until authenticated and authorized.

Therefore:

Trustdefault=0Trust_{default}=0

and:

$\text{Trust}_{\text{granted}} = \text{Authenticated} \wedge \text{Authorized} \wedge \text{PolicyCompliant}$
 $\text{Trust}_{\text{granted}} = \text{Authenticated} \wedge \text{Authorized} \wedge \text{PolicyCompliant}$

A valid identity does not automatically imply valid authorization.

9.11 CLIENT DEVELOPER THREAT MODEL

The architecture assumes that a participating client developer may attempt to:

- alter checkout configuration;
- remove the protocol charge;
- redirect settlement;
- spoof synchronization status;
- modify adapter configuration;
- suppress protocol events;
- replay events;
- bypass validation;
- disable telemetry.

The protocol must therefore ensure that client-side software cannot become the ultimate authority over the economic or interoperability state.

9.12 FRONTEND TRUST EXCLUSION

The frontend is explicitly excluded from the settlement trust boundary.

A frontend variable such as:

```
protocolFeePaid = true
```

has no cryptographic authority.

Likewise:

```
networkSynchronized = true
```

is not accepted as authoritative evidence.

The authoritative sources remain:

BlockchainState BlockchainState ContractState ContractState
VerifiedProtocolState VerifiedProtocolState

and approved infrastructure attestations.

9.13 CONFIGURATION TAMPERING

Production configuration shall be integrity-protected.

Critical parameters include:

SplitterAddressSplitterAddress ChainIDChainID CodeHashCodeHash
TreasuryAddressTreasuryAddress VaultAddressVaultAddress RoutePolicyRoutePolicy

Unauthorized modification produces:

ConfigurationIntegrity=FALSEConfigurationIntegrity=FALSE

and prevents the affected adapter from entering an authorized operating state.

9.14 SECURITY BLAST-RADIUS CONTAINMENT

A compromised client adapter must not automatically imply:

TreasuryCompromiseTreasuryCompromise

A compromised frontend must not imply:

ProtocolCompromiseProtocolCompromise

A compromised treasury signer must not imply:

ClinicalDatabaseCompromiseClinicalDatabaseCompromise

The architecture is therefore designed around compartmentalized failure domains.

9.15 GOVERNANCE PRINCIPLE

Haven governance controls the protocol's rules.

It does not become the custodian of every participating institution's clinical database.

This distinction is fundamental to the stateless coordination model

THREAT MODEL AND ADVERSARIAL CONTROL FRAMEWORK

10.1 THREAT MODEL OBJECTIVE

The Haven threat model assumes that compromise may occur at any layer.

The system therefore evaluates:

Threat→AttackSurface→Control→ResidualRisk
Threat \rightarrow AttackSurface \rightarrow Control \rightarrow ResidualRisk

The objective is not to assume perfect infrastructure.

The objective is to ensure that failure of one layer does not automatically compromise the entire protocol.

10.2 ATTACK SURFACE

Principal attack surfaces include:

ClientApplication ClientApplication Checkout Checkout EdgeAdapter EdgeAdapter
RPCInfrastructure RPCInfrastructure SmartContract SmartContract
TransportProvider TransportProvider Treasury Treasury IdentityLayer IdentityLayer

and:

OperationalGovernance OperationalGovernance

10.3 PAYMENT BYPASS ATTACK

Attack:

Client→AlternativePaymentDestination Client \rightarrow AlternativePaymentDestination

instead of:

Client→Splitter Client \rightarrow Splitter

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

The protocol must not treat the client's declaration of payment as evidence.

Instead:

$$\text{ExpectedSettlementEndpoint} = \text{CExpectedSettlementEndpoint} = \text{C_S}$$

and:

$$\text{ObservedSettlementEndpoint} = \text{CSObservedSettlementEndpoint} = \text{C_S}$$

must be verified.

If:

$$\text{Observed} \neq \text{Expected}$$

the event fails settlement authorization.

10.4 CONTRACT SUBSTITUTION ATTACK

An attacker may attempt to replace the canonical contract with a malicious contract.

The adapter verifies:

$$\text{Address} = \text{Address} \wedge \text{ChainID} = \text{ChainID}$$

and:

$$\text{CodeHash} = \text{CodeHash}$$

The authorization condition is:

$$\text{ContractValid} = \text{AddressMatch} \wedge \text{ChainMatch} \wedge \text{CodeHashMatch} \wedge \text{ContractValid} = \text{AddressMatch} \wedge \text{ChainMatch} \wedge \text{CodeHashMatch}$$

10.5 SYNCHRONIZATION SPOOFING

A malicious client may attempt to report:

$$\text{Sync} = \text{TRUE} \wedge \text{Sync} = \text{TRUE}$$

without satisfying the protocol's settlement requirements.

The adapter therefore independently queries authoritative state.

Thus:

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

$\text{ClientSyncClaim} \Rightarrow \text{ProtocolSyncClientSyncClaim} \not\Rightarrow \text{ProtocolSync}$

10.6 RPC MANIPULATION

RPC endpoints are treated as information sources, not trusted authorities.

Where practical, the adapter may query multiple independent RPC endpoints.

For observations:

$R_1, R_2, \dots, R_n$ $R_1, R_2, \dots, R_n$

the system may establish:

$\text{Consensus}(R_1, \dots, R_n)$ $\text{Consensus}(R_1, \dots, R_n)$

before accepting a critical state observation.

An RPC disagreement enters:

UNKNOWN UNKNOWN

rather than:

TRUE TRUE

10.7 REPLAY ATTACK

An attacker may capture a valid Haven event and retransmit it.

The protocol defeats this using:

EventID EventID Nonce Nonce Domain Domain

and:

ConsumedState ConsumedState

The acceptance condition includes:

ReplayCheck=TRUE ReplayCheck=TRUE

before state transition.

10.8 MESSAGE FORGERY

A forged event must fail source authentication.

$\text{VerifySignature}(E) = \text{FALSE}$

produces:

REJECT

The destination must never infer source authority from payload contents alone.

10.9 TRANSPORT COMPROMISE

If an interoperability transport delivers an altered or unauthorized message:

$\text{TransportDelivery} = \text{TRUE}$

does not imply:

$\text{HavenAcceptance} = \text{TRUE}$

The destination independently verifies the Haven message commitment and authorization predicates.

10.10 TREASURY COMPROMISE

Treasury risk is isolated through multisignature custody.

For threshold:

$m/nm/n$

a compromised signer does not automatically provide withdrawal authority unless:

$\text{CompromisedSigners} \geq m$

Additional operational controls may include signer separation, transaction review, spending limits, and hardware-backed authentication.

10.11 SMART-CONTRACT EXPLOIT

Potential contract vulnerabilities include:

- reentrancy;
- arithmetic errors;
- incorrect access control;
- unsafe token handling;
- denial of service;
- destination substitution;
- accounting inconsistencies.

The contract must therefore be subjected to formal testing and independent security review before economic activation.

10.12 DENIAL-OF-SERVICE

An attacker may attempt to prevent protocol events from progressing.

Haven shall distinguish:

AvailabilityFailureAvailabilityFailure

from:

DataIntegrityFailureDataIntegrityFailure

A temporary transport failure should place events into controlled asynchronous queues rather than corrupting the underlying clinical state.

10.13 FAIL-CLOSED SECURITY STATE

Where authorization cannot be established:

Authorization=UNKNOWNAuthorization=UNKNOWN

shall not be converted into:

Authorization=TRUEAuthorization=TRUE

For protocol-critical cross-network transmission:

UNKNOWN $\rightarrow$ BLOCK UNKNOWN $\rightarrow$ BLOCK

with the event preserved for reconciliation where policy permits.

10.14 THREAT MODEL INVARIANT

The central security invariant is:

$\text{Compromise}(\text{Layer}_i) \Rightarrow \text{Compromise}(\text{All Layers})$
 $\text{Compromise}(\text{Layer}_i) \not\Rightarrow \text{Compromise}(\text{All Layers})$

The architecture is considered resilient only where individual-layer compromise remains bounded by explicit trust boundaries.

STATELESS COORDINATION AND FORMAL STATE MODEL

11.1 PROTOCOL STATE DEFINITION

Haven does not require ownership of the underlying clinical database to coordinate a state transition.

For an event EE, define the protocol state as:

$SH(E) = \{ID, Commitment, Source, Destination, Nonce, Policy, SettlementReference, Timestamp\}$
 $H(E) = \{ID, Commitment, Source, Destination, Nonce, Policy, SettlementReference, Timestamp\}$

The protocol state describes the **coordination condition**, not the complete clinical record.

Therefore:

$SH \neq ClinicalRecord$

11.2 STATE COMMITMENT

Let the normalized event payload be:

PE_E

Haven computes:

$CE = H(PE)$

where HH is a cryptographically secure hash function.

The protocol records or transmits:

CEC_E

rather than requiring the entire clinical payload to become centralized protocol state.

The integrity property is:

$PE = PE' \Rightarrow H(PE) = H(PE') \wedge P_E = P'_E \Rightarrow H(P_E) = H(P'_E)$

while, assuming collision resistance:

$$PE \neq PE' \Rightarrow \Pr[H(PE) = H(PE')] \approx 0 \quad P_E \neq P'_E \Rightarrow \Pr[H(P_E) = H(P'_E)] \approx 0$$

11.3 STATE TRANSITION FUNCTION

Define the Haven transition function:

$$F(S, E) \rightarrow S' \quad F(S, E) \rightarrow S'$$

A valid transition satisfies:

$$S' = F(S, E) \quad S' = F(S, E)$$

The transition does not require Haven to permanently store every input used to construct EE.

The authoritative clinical source may retain the underlying record while Haven retains the minimum coordination evidence required to establish the transition.

11.4 DISTRIBUTED STATE OBSERVATION

Assume two independent institutions:

AA

and:

BB

Institution AA possesses:

RAR_A

Institution BB possesses:

RBR_B

Haven need not possess either complete record.

Instead:

Commit(RA)Commit(R_A)

and:

Commit(RB)Commit(R_B)

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

can establish verifiable relationships between independently governed records.

The coordination layer therefore becomes:

$$A \leftrightarrow \text{Haven} \leftrightarrow B \quad \text{BA} \rightarrow \text{Haven} \rightarrow B$$

without becoming:

$$A+B \rightarrow \text{CentralDatabase} \quad A+B \rightarrow \text{CentralDatabase}$$

11.5 PROOF OF STATE CONTINUITY

For consecutive states:

$$S_i$$

and:

$$S_{i+1}$$

define:

$$C_i = H(C_{i-1} \parallel \text{EventID}_i \parallel \text{EventCommitment}_i \parallel \text{Nonce}_i) \quad C_i = H(C_{i-1} \parallel \text{EventID}_i \parallel \text{EventCommitment}_i \parallel \text{Nonce}_i)$$

The resulting chain is:

$$C_0 \rightarrow C_1 \rightarrow C_2 \rightarrow \dots \rightarrow C_n \quad C_0 \rightarrow C_1 \rightarrow C_2 \rightarrow \dots \rightarrow C_n$$

A modification to an earlier event changes:

$$C_i$$

which propagates through subsequent commitments.

Therefore historical tampering becomes detectable.

11.6 NON-CUSTODIAL DATA PROPERTY

Let:

$$DCD_C$$

represent the complete clinical dataset.

Let:

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

DHD_H

represent Haven's protocol-level data.

The architectural objective is:

$DH \not\subseteq DCD_H \setminus \text{subseteq } D_C$

and:

$DC \subseteq DHD_C \setminus \text{subseteq } D_H$

Haven therefore does not require a complete replica of the clinical dataset to perform protocol coordination.

11.7 MINIMUM NECESSARY STATE

For each event:

EE

the protocol seeks the smallest state vector sufficient to verify:

IdentityIdentity IntegrityIntegrity AuthorizationAuthorization OrderingOrdering
SettlementSettlement

and:

DestinationDestination

Thus:

$|SH| \ll |DC| |S_H| \setminus |D_C|$

for a typical clinical deployment.

This is a design objective rather than an assumption that all implementations will have identical storage requirements.

11.8 VERIFICATION WITHOUT CENTRAL STORAGE

A verifier possessing:

SHS_H

and the appropriate authorized evidence can determine whether:

$\text{ValidTransition}(E) = \text{TRUE}$

without receiving the entire underlying clinical database.

Therefore:

$\text{Verification} \Rightarrow \neg \text{CentralizedDataCustodyVerification} \rightarrow \text{CentralizedDataCustody}$

This is the principal mathematical basis of the stateless coordination model.

11.9 PROOF OF NON-BYPASSABLE SETTLEMENT CONDITION

For a billable event:

E_b

define:

$\text{Authorization}(E_b) = \text{ProtocolValid}(E_b) \wedge \text{SettlementValid}(E_b)$
 $\text{Authorization}(E_b) = \text{ProtocolValid}(E_b) \wedge \text{SettlementValid}(E_b)$

Then:

$\text{SettlementValid}(E_b) = \text{FALSE} \Rightarrow \text{Authorization}(E_b) = \text{FALSE}$
 $\neg \text{SettlementValid}(E_b) \rightarrow \text{Authorization}(E_b) = \text{FALSE}$

assuming the adapter correctly enforces the protocol authorization boundary.

Therefore a client application cannot create an authorized Haven event merely by modifying its local payment logic.

11.10 STATE MACHINE INVARIANT

The principal invariant is:

$\neg \text{UnauthorizedState} \rightarrow \text{AuthorizedState}$
 $\boxed{\neg \text{UnauthorizedState} \rightarrow \text{AuthorizedState}}$

unless the required protocol predicates are satisfied.

Similarly:

$\text{ConsumedEvent} \rightarrow \text{NewValidEvent}$
 $\boxed{\text{ConsumedEvent} \rightarrow \text{NewValidEvent}}$

through replay alone.

These invariants define the core security properties of the Haven state machine.

STATELESS EVENT TOKENIZATION AND PRODUCTION ACCEPTANCE

12.1 EVENT TOKEN MODEL

Haven event tokenization does not imply creation of a speculative financial asset.

A protocol event token is an **identity and state-reference primitive**.

Define:

$$TE = H(\text{ProtocolID} \parallel \text{EventID} \parallel \text{Source} \parallel \text{Destination} \parallel \text{Nonce} \parallel \text{Commitment})$$
$$T_E = H(\text{ProtocolID} \parallel \text{EventID} \parallel \text{Source} \parallel \text{Destination} \parallel \text{Nonce} \parallel \text{Commitment})$$

The token represents the existence and identity of a protocol event.

It does not represent ownership of the underlying clinical record.

12.2 TOKEN UNIQUENESS

For an event:

$$E_i$$

the protocol requires:

$$T_i \neq T_j \quad T_i \neq T_j$$

for distinct valid events under the same domain.

Uniqueness is enforced through the combination of:

$$\text{EventID} \parallel \text{EventID} \parallel \text{Nonce} \parallel \text{Nonce} \parallel \text{Domain} \parallel \text{Domain}$$

and:

$$\text{Commitment} \parallel \text{Commitment}$$

12.3 TOKEN LIFECYCLE

The event token follows:

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

Created→Committed→Settled→Routed→Consumed
Created \rightarrow Committed \rightarrow
Settled \rightarrow Routed \rightarrow Consumed

A token may subsequently enter:

ExpiredExpired

or:

ReconciledReconciled

depending on policy.

12.4 TOKEN NON-OWNERSHIP

The event token does not confer:

ClinicalOwnershipClinicalOwnership PatientOwnershipPatientOwnership
DatabaseOwnershipDatabaseOwnership

or:

FinancialAssetOwnershipFinancialAssetOwnership

unless a separate legally and technically defined instrument explicitly establishes such rights.

The token is therefore a coordination primitive.

PRODUCTION ACCEPTANCE CRITERIA

The Haven Core Protocol Blueprint shall be considered technically complete only when the following conditions are satisfied.

13.1 ARCHITECTURAL ACCEPTANCE

MacroArchitecture=DEFINEDMacroArchitecture=DEFINED
EdgeBoundary=DEFINEDEdgeBoundary=DEFINED
SettlementBoundary=DEFINEDSettlementBoundary=DEFINED
RoutingBoundary=DEFINEDRoutingBoundary=DEFINED
GovernanceBoundary=DEFINEDGovernanceBoundary=DEFINED

13.2 SMART-CONTRACT ACCEPTANCE

ContractAddress=VERIFIEDContractAddress=VERIFIED
CodeHash=VERIFIEDCodeHash=VERIFIED
DestinationAddresses=VERIFIEDDestinationAddresses=VERIFIED
AccountingInvariant=PASSAccountingInvariant=PASS
ReplayProtection=PASSReplayProtection=PASS SecurityReview=PASSSecurityReview=PASS

13.3 INTEROPERABILITY ACCEPTANCE

At least one production-approved transport path must satisfy:

RouteApprovedRouteApproved SourceAuthenticatedSourceAuthenticated
DestinationAuthenticatedDestinationAuthenticated ReplayProtectionReplayProtection
FinalityPolicyFinalityPolicy

and:

FailureHandlingFailureHandling

13.4 EDGE ACCEPTANCE

The Edge Adapter must demonstrate:

EventValidationEventValidation SettlementVerificationSettlementVerification
QueuePersistenceQueuePersistence IdempotencyIdempotency CircuitBreakingCircuitBreaking

and:

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

ReconciliationReconciliation

under controlled test conditions.

13.5 ECONOMIC ACCEPTANCE

For protocol fee:

FF

the settlement invariant must hold:

$$FT + FV = F \quad \boxed{F_T + F_V = F}$$

No client-side declaration may substitute for on-chain settlement evidence.

13.6 STATELESSNESS ACCEPTANCE

The architecture must demonstrate:

$ClinicalData \rightarrow MandatoryCentralReplicaClinicalData \not\rightarrow MandatoryCentralReplica$

while still permitting:

$EventVerificationEventVerification \rightarrow StateCoordinationStateCoordination$

and:

$CrossNetworkSynchronizationCrossNetworkSynchronization$

through cryptographic commitments and authorized state references.

13.7 FINAL SYSTEM INVARIANT

The complete Haven architecture can therefore be reduced to:

$LocalClinicalState + ProtocolCommitment + EconomicSettlement + TransportVerification \rightarrow AuthorizedCoordination \quad \boxed{LocalClinicalState + ProtocolCommitment + EconomicSettlement + TransportVerification \rightarrow AuthorizedCoordination}$

without requiring:

$Haven = CentralClinicalDatabaseHaven = CentralClinicalDatabase$

and without allowing:

ClientApplication=ProtocolAuthorityClientApplication = ProtocolAuthority

CORE PROTOCOL BLUEPRINT CLOSURE

The Haven Protocol is defined as a stateless coordination infrastructure positioned beneath fragmented healthcare software environments.

Its principal architectural properties are:

StatelessCoordinationStatelessCoordination

AsynchronousEventProcessingAsynchronousEventProcessing

CryptographicStateCommitmentCryptographicStateCommitment

EVMSettlementEVMSettlement

MultiChainTransportAbstractionMultiChainTransportAbstraction

ZeroTrustVerificationZeroTrustVerification DistributedCustodyDistributedCustody

and:

InstitutionalDataSovereigntyInstitutionalDataSovereignty

The protocol coordinates.

The participating institution retains its clinical records.

The blockchain independently verifies the designated economic state.

The transport layer carries protocol messages.

The Edge Adapter enforces the boundary.

No individual application component constitutes the complete authority of the system.

HAVEN=COORDINATIONWITHOUTCENTRALIZEDCLINICALOWNERSHIP\boxed{
HAVEN = COORDINATION WITHOUT CENTRALIZED CLINICAL OWNERSHIP }

END OF DOCUMENT 01 — CORE PROTOCOL BLUEPRINT

CANONICAL PRODUCTION CONFIGURATION REGISTRY

The following registry constitutes the controlled configuration boundary for production deployment.

15.1 POLYGON SETTLEMENT IDENTIFIERS

Parameter	Production Configuration
Network	Polygon Mainnet
Chain ID	137
Settlement Asset	USDC
Canonical Haven Polygon Address	0xC02E7Fe6AAfFF15F0bc4636B53E6d9D20898c4ac
Address Classification	To be independently verified at deployment
Contract Code Hash	To be recorded if applicable
Deployment Transaction	To be recorded if applicable
Deployment Block	To be recorded if applicable

The canonical Haven Polygon address is the sole designated settlement and treasury destination for the baseline deployment.

No separate Haven Treasury Safe Address or Haven Vault Address shall be required by the baseline configuration.

15.2 CRITICAL DEPLOYMENT DISTINCTION

The production payment architecture requires a smart-contract endpoint:

CSC_S

and a wallet destination:

DD

These are fundamentally different blockchain objects.

The supplied address represents:

DUSDCD_{USDC}

until a deployed contract establishes otherwise.

The final production path must therefore be explicitly defined as either:

Checkout→SplitterContract→USDC→TreasurySafe/VaultCheckout \rightarrow SplitterContract
\rightarrow USDC \rightarrow TreasurySafe/Vault

or another formally approved settlement sequence.

A wallet address must never be substituted for the immutable Splitter Contract merely because both exist on Polygon.

15.3 PROTOCOL CONFIGURATION OBJECT

The canonical deployment object is:

ConfigH={Network,Asset,Splitter,Treasury,Vault,Routes,SecurityPolicy} Config_H= \{
Network, Asset, Splitter, Treasury, Vault, Routes, SecurityPolicy \}

No production adapter shall rely upon undocumented addresses embedded inside application source code.

15.4 ENVIRONMENT SEPARATION

Configuration shall exist independently for:

DevelopmentDevelopment TestnetTestnet StagingStaging

and:

ProductionProduction

Production addresses must never be inherited automatically from development configuration.

15.5 IMMUTABLE IDENTIFIERS

Once production deployment is activated, the following become pinned infrastructure identifiers:

SplitterAddressSplitterAddress ChainIDChainID CodeHashCodeHash
SettlementAssetSettlementAsset

and approved destination addresses.

Any change requires a formal deployment or migration procedure.

15.6 CONFIGURATION INTEGRITY

The adapter shall verify that its loaded production configuration corresponds to the authorized deployment manifest.

$\text{ConfigObserved} = \text{ConfigAuthorized}$

must evaluate:

TRUE

before the adapter enters an authorized operating state.

15.7 FINAL ADDRESS INSERTION RULE

The canonical Polygon address for the initial Haven deployment is:

0xC02E7Fe6AAFF15F0bc4636B53E6d9D20898c4ac

This address shall be used consistently wherever the baseline architecture requires the designated Haven Polygon settlement and treasury destination.

Its classification as an externally owned account or smart contract shall be independently verified during deployment.

No additional Haven treasury or vault address is required by the baseline configuration.

The address shall not be represented as an immutable smart-contract address unless the deployed on-chain code independently verifies that classification.

address from being incorrectly represented as a deployed protocol contract.

15.8 CONFIGURATION AUTHORITY

The registry shall be maintained under controlled deployment governance.

Client developers must not be able to modify production settlement configuration through:

Frontend Checkout SDK

or:

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

ClientAPIClientAPI

The registry is infrastructure configuration, not application input.

PRODUCTION DEPLOYMENT ACCEPTANCE AND FINAL ARCHITECTURAL INVARIANTS

16.1 PRODUCTION DEPLOYMENT ACCEPTANCE CHECKLIST

Production activation shall occur only after the following infrastructure, security, settlement, and interoperability conditions have been independently verified.

SMART CONTRACT & PROTOCOL INTEGRITY

ContractDeployment=VERIFIEDContractDeployment=VERIFIED
ContractCodeIntegrity=VERIFIEDContractCodeIntegrity=VERIFIED
InterfaceCompliance=VERIFIEDInterfaceCompliance=VERIFIED
SettlementLogic=TESTEDSettlementLogic=TESTED
CircuitBreakerLogic=TESTEDCircuitBreakerLogic=TESTED
ReplayProtection=TESTEDReplayProtection=TESTED

Where a smart contract is used for settlement execution, its deployed bytecode, address, chain, and interface shall be independently verified before production traffic is permitted.

POLYGON SETTLEMENT

Network=Polygon MainnetNetwork=Polygon\ Mainnet ChainID=137ChainID=137
SettlementAsset=USDCSettlementAsset=USDC

The sole canonical Haven Polygon settlement and treasury destination is:

0xC02E7Fe6AAfFF15F0bc4636B53E6d9D20898c4ac

The following conditions shall be verified:

CanonicalAddress=VERIFIEDCanonicalAddress=VERIFIED
AddressClassification=VERIFIEDAddressClassification=VERIFIED
SettlementPath=TESTEDSettlementPath=TESTED
TransactionCorrelation=TESTEDTransactionCorrelation=TESTED
SettlementReconciliation=TESTEDSettlementReconciliation=TESTED

The canonical address shall not be replaced through client-side configuration, frontend variables, checkout parameters, or Edge Adapter configuration.

If settlement execution is performed by a smart contract, engineers shall explicitly document the relationship between the executing contract and the canonical settlement destination.

EDGE ADAPTER

ContractPinning=ACTIVEContractPinning=ACTIVE
RPCVerification=ACTIVERPCVerification=ACTIVE
SynchronizationVerification=ACTIVESynchronizationVerification=ACTIVE
CircuitBreaker=TESTEDCircuitBreaker=TESTED
QueueRecovery=TESTEDQueueRecovery=TESTED

The Edge Adapter shall reject unauthorized settlement destinations and shall prevent cross-network coordination where required protocol authorization cannot be established.

EVENT PROCESSING

EventNormalization=TESTEDEventNormalization=TESTED
EventCommitment=TESTEDEventCommitment=TESTED
EventIdempotency=TESTEDEventIdempotency=TESTED
StateTransitionVerification=TESTEDStateTransitionVerification=TESTED
DuplicateEventRejection=TESTEDDuplicateEventRejection=TESTED

Every protocol event shall possess a deterministic identity sufficient to prevent unintended duplicate settlement or duplicate state transition.

CROSS-CHAIN INTEROPERABILITY

RouteRegistry=VERIFIEDRouteRegistry=VERIFIED
TransportSelection=DEFINEDTransportSelection=DEFINED
LayerZeroIntegration=VALIDATEDLayerZeroIntegration=VALIDATED
CCIPIntegration=VALIDATEDCCIPIntegration=VALIDATED
DestinationValidation=ACTIVEDestinationValidation=ACTIVE
ReplayProtection=ACTIVEReplayProtection=ACTIVE
ExecutionBounds=ACTIVEExecutionBounds=ACTIVE

Cross-chain transport shall remain subordinate to the Haven protocol semantic layer.

OBSERVABILITY & FORENSIC RECONCILIATION

The production environment shall provide sufficient telemetry to reconstruct:

Event→Commitment→Settlement→Transport→Destination
Event \rightarrow Commitment
\rightarrow Settlement \rightarrow Transport \rightarrow Destination

using transaction hashes, block references, event logs, timestamps, event identifiers, and applicable transport metadata.

PRODUCTION ACTIVATION CONDITION

Production traffic shall be authorized only where:

$\text{ProtocolIntegrity} \wedge \text{SettlementIntegrity} \wedge \text{EdgeIntegrity} \wedge \text{TransportIntegrity} = \text{TRUE}$
 $\text{ProtocolIntegrity} \wedge \text{SettlementIntegrity} \wedge \text{EdgeIntegrity} \wedge \text{TransportIntegrity} = \text{TRUE}$

If any mandatory condition evaluates to false:

$\text{ProductionActivation} = \text{REJECTED}$

The system shall fail closed rather than silently operate under an unverified settlement or transport configuration.

16.2 MASTER ARCHITECTURAL INVARIANTS

The Haven Protocol shall preserve the following invariants throughout production operation.

INVARIANT I — PAYMENT AUTHORITY

A client-side payment confirmation shall not independently constitute protocol settlement authority.

$\text{ClientPaymentClaim} \Rightarrow \text{SettlementValid}$
 $\text{ClientPaymentClaim} \wedge \neg \text{SettlementValid} \Rightarrow \text{SettlementRejected}$

Settlement validity shall be determined through the authorized protocol settlement path.

INVARIANT II — CANONICAL DESTINATION

All Haven Polygon settlement references within the baseline deployment shall resolve to:

0xC02E7Fe6AAfFF15F0bc4636B53E6d9D20898c4ac

Therefore:

$\text{ObservedDestination} \neq \text{CanonicalDestination} \Rightarrow \text{SettlementRejected}$
 $\text{ObservedDestination} \neq \text{CanonicalDestination} \Rightarrow \text{SettlementRejected}$

INVARIANT III — CONTRACT INTEGRITY

A contract address shall not be considered authoritative solely because it is supplied by a client application.

$\text{UnknownContract} \Rightarrow \neg \text{ProtocolAuthorizationUnknownContract} \rightarrow \text{ProtocolAuthorization}$

Contract identity shall be established through the canonical deployment configuration and applicable on-chain verification.

INVARIANT IV — REPLAY RESISTANCE

A previously consumed event shall not produce a second valid protocol state transition.

$\text{ConsumedEvent} \Rightarrow \neg \text{SecondValidTransitionConsumedEvent} \rightarrow \text{SecondValidTransition}$

Therefore:

$\text{EventIDused} = \text{TRUE} \Rightarrow \text{Replay} = \text{REJECTED} \mid \text{EventID}_{\text{used}} = \text{TRUE} \rightarrow \text{Replay} = \text{REJECTED}$

INVARIANT V — DATA SOVEREIGNTY

Protocol coordination shall not constitute ownership of participating clinical databases.

$\text{ProtocolCoordination} \Rightarrow \neg \text{CentralClinicalOwnershipProtocolCoordination} \rightarrow \text{CentralClinicalOwnership}$

Haven shall coordinate state transitions without requiring centralized ownership of the underlying clinical records.

INVARIANT VI — TRANSPORT INDEPENDENCE

The semantic definition of a Haven event shall remain independent of the selected interoperability transport.

$\text{TransportReplacement} \Rightarrow \neg \text{ProtocolSemanticChangeTransportReplacement} \rightarrow \text{ProtocolSemanticChange}$

LayerZero, Chainlink CCIP, or another authorized transport mechanism shall function as a delivery substrate rather than as the owner of Haven event semantics.

INVARIANT VII — FAIL CLOSED

Where mandatory authorization, contract verification, destination verification, or synchronization state cannot be established:

$\text{Authorization}=\text{UNKNOWN} \Rightarrow \text{ProtocolAuthorization}=\text{FALSE}$
 $\text{Authorization}=\text{UNKNOWN} \wedge \text{ProtocolAuthorization}=\text{FALSE}$

and:

$\text{ProtocolAuthorization}=\text{FALSE} \Rightarrow \text{ProtectedOperation}=\text{BLOCKED}$
 $\text{ProtocolAuthorization}=\text{FALSE} \wedge \text{ProtectedOperation}=\text{BLOCKED}$

The protocol shall prefer interruption over execution against an unverified infrastructure state.

INVARIANT VIII — EDGE ENFORCEMENT

The Edge Adapter shall enforce protocol authorization independently of client application logic.

$\text{ClientConfiguration} \Rightarrow \text{ProtocolAuthorization}$
 $\text{ClientConfiguration} \not\Rightarrow \text{ProtocolAuthorization}$

A client shall not be capable of establishing protocol authorization merely by modifying local checkout or application variables.

INVARIANT IX — SETTLEMENT DESTINATION IMMUTABILITY

The baseline production destination shall remain fixed unless a formally governed protocol migration is executed.

$\text{UnauthorizedAddressChange} \Rightarrow \text{REJECT}$
 $\text{UnauthorizedAddressChange} \wedge \text{REJECT}$

The canonical Haven Polygon address is:

0xC02E7Fe6AAFF15F0bc4636B53E6d9D20898c4ac

INVARIANT X — STATE WITHOUT CENTRAL CLINICAL STORAGE

The protocol shall derive coordination state from event commitments, transaction evidence, and verifiable state transitions rather than requiring a centralized Haven-owned clinical database.

$\text{CoordinationState} = f(\text{EventCommitment}, \text{SettlementProof}, \text{TransportProof})$
 $\text{CoordinationState} = f(\text{EventCommitment}, \text{SettlementProof}, \text{TransportProof})$

Therefore:

CentralClinicalDatabaseOwnership $\Rightarrow$ ProtocolOperationCentralClinicalDatabaseOwnership
 $\neg$ CentralClinicalDatabaseOwnership $\Rightarrow$ ProtocolOperation

INVARIANT XI — ECONOMIC EXECUTION BINDING

Where a protocol event carries a mandatory economic obligation, settlement verification shall form part of the event authorization boundary.

RequiredSettlement $\wedge$ SettlementVerified $\Rightarrow$ EventAuthorizationRequiredSettlement $\wedge$ SettlementVerified $\Rightarrow$ EventAuthorization

Conversely:

RequiredSettlement $\wedge$ SettlementNotVerified $\Rightarrow$ EventAuthorization=FALSERequiredSettlement $\wedge$ SettlementNotVerified $\Rightarrow$ EventAuthorization=FALSE

INVARIANT XII — AUDITABILITY

Every economically or operationally material protocol transition shall produce sufficient cryptographic and transactional evidence for independent reconstruction.

MaterialStateTransition $\Rightarrow$ VerifiableEvidenceMaterialStateTransition $\Rightarrow$ VerifiableEvidence

The protocol shall therefore preserve:

EventID+Commitment+TransactionEvidence+StateTransitionEventID + Commitment + TransactionEvidence + StateTransition

as the minimum forensic relationship required for reconstruction.

MASTER INVARIANT

The Haven Protocol shall preserve the following overarching condition:

UnauthorizedConfiguration $\Rightarrow$ AuthorizedProtocolExecution $\wedge$ UnauthorizedConfiguration $\Rightarrow$ AuthorizedProtocolExecution

and:

VerifiedState+AuthorizedSettlement+ValidTransport $\Rightarrow$ PermittedCoordinationVerifiedState + AuthorizedSettlement + ValidTransport $\Rightarrow$ PermittedCoordination

The protocol shall therefore operate as a deterministic coordination and verification infrastructure layer beneath fragmented healthcare systems without requiring centralized ownership of their underlying clinical data.

16.3 MASTER SYSTEM EQUATION

The Haven architecture can be represented as:

$$E \rightarrow \text{Normalize} \rightarrow CE \rightarrow \text{Settlement} \rightarrow SE \rightarrow \text{Verify} \rightarrow AE \rightarrow \text{Route} \rightarrow DE \rightarrow \boxed{E \rightarrow \text{Normalize} \rightarrow C_E \rightarrow \text{Settlement} \rightarrow S_E \rightarrow \text{Verify} \rightarrow A_E \rightarrow \text{Route} \rightarrow D_E}$$

where:

E =LocalEvent E =LocalEvent CE =EventCommitment C_E =EventCommitment
 SE =SettlementState S_E =SettlementState AE =AuthorizationState A_E =AuthorizationState

and:

DE =DestinationEvent D_E =DestinationEvent

The protocol does not require ownership of the complete clinical payload at any stage.

16.4 FINAL ARCHITECTURAL MODEL

The completed Haven system is:

$$\text{ClinicalSystems} \rightarrow \text{EdgeAdapters} \rightarrow \text{StatelessProtocol} \rightarrow \text{EVMSettlement} \rightarrow \text{MultiChainRouting} \rightarrow \text{EdgeAdapters} \rightarrow \text{ClinicalSystems} \rightarrow \boxed{\text{ClinicalSystems} \rightarrow \text{EdgeAdapters} \rightarrow \text{StatelessProtocol} \rightarrow \text{EVMSettlement} \rightarrow \text{MultiChainRouting} \rightarrow \text{EdgeAdapters} \rightarrow \text{ClinicalSystems}}$$

with:

$\text{Blockchain} = \text{EconomicProof} + \text{StateAttestation}$ $\text{Blockchain} = \text{EconomicProof} + \text{StateAttestation}$

rather than:

$\text{Blockchain} = \text{ClinicalDatabase}$ $\text{Blockchain} = \text{ClinicalDatabase}$

16.5 CORE PROTOCOL POSITION

Haven is therefore defined as infrastructure that coordinates independently governed healthcare systems through deterministic event commitments, asynchronous translation, cryptographically verifiable settlement conditions, and transport-independent routing.

The protocol's authority derives from:

CodeCode CryptographicProofCryptographicProof VerifiedStateVerifiedState

and:

InstitutionalPolicyInstitutionalPolicy

not from unilateral ownership of participating clinical databases.

16.6 DOCUMENT STATUS

DOCUMENT: Core Protocol Blueprint

IDENTIFIER: CPB-001

VERSION: 2.0

STATUS: Architectural Baseline

NETWORK: Polygon / Multi-Chain Ready

SETTLEMENT ASSET: USDC

PRIMARY WALLET RECORDED: 0xC02E7Fe6AAfFF15F0bc4636B53E6d9D20898c4ac

SPLITTER: Pending Production Deployment

TREASURY SAFE: Pending Production Deployment

HAVEN VAULT: Pending Production Deployment

CORE PROTOCOL BLUEPRINT — COMPLETE